Tampere University

Syed Babar Ali Rizvi

A SYSTEMATIC MAPPING STUDY ON THE USE OF LLMS FOR REQUIREMENTS TRACEABILITY LINK DISCOVERY

Master's Thesis
Faculty of Information Technology and Communication Sciences
Supervisor: Zheying Zhang
July 2025

ABSTRACT

Syed Babar Ali Rizvi: A systematic mapping study on the use of large language models for requirements traceability link discovery
Master's Thesis
Tampere University
Master's Degree Programme in Computing Sciences
July 2025Degree Programme

Requirements traceability helps developers and stakeholders understand how different software artifacts including requirements, design models, source code, and test cases are connected. It is considered important for change impact analysis, test coverage verification, and quality assurance, especially in safety-critical domains. However, creating and maintaining this trace links is often time-consuming and incomplete in practice, which has led to long-standing interest in automating the process through techniques known as Traceability Link Recovery (TLR).

Earlier approaches to TLR have relied on information retrieval, rule-based heuristics, and supervised learning. While these methods can identify basic relationships, they often struggle to handle language variability and project-specific semantics. In recent years, research has begun to explore whether Large Language Models (LLMs) can improve traceability by understanding natural language more deeply. Some studies report encouraging results, suggesting that LLMs may identify trace links even when textual overlap is minimal. Hybrid methods, such as retrieval-augmented generation, have also been investigated to address the limitations of LLM input size and generalization.

This thesis presents a systematic mapping study of **132 peer-reviewed studies (2015–2025)** that investigate the use of LLMs for requirements traceability link recovery (TLR). The study classifies techniques into prompting, fine-tuning, and retrieval-augmented generation (RAG), and analyzes their use across various traceability tasks (e.g., requirements-to-code, requirements-to-test). Key findings reveal that 44% of studies use prompting, 29% fine-tuning, and 13% RAG, with requirements-to-code being the most common traceability direction.

While LLM-based methods outperform traditional approaches in many cases, challenges such as input length constraints, hallucinations, and explainability remain. This mapping highlights emerging tools, datasets, and industrial applications, offering actionable insights for researchers and practitioners. The results underscore LLMs' growing potential to augment traceability in complex, evolving software systems.

Keywords: Requirements Traceability, Large Language Models (LLM), Traceability Link Recovery, Systematic Mapping Study, Retrieval-Augmented Generation (RAG)

1. USE OF AI IN THESIS

I have utilised AI tools in my thesis:

☐ No
× Yes

The AI tools utilised in my thesis and their purposes are described below:

Names and versions of AI tools:
1. Quadratic (2024 version)
2. OpenAI ChatGPT (GPT-4, 2023–2025 releases)

Purpose of using AI tools:
- **Quadratic** was used for organizing, analyzing, and visualizing mapping study data in tabular and graphical form. It supported filtering, categorization, and generating summary plots.
- **ChatGPT** was used during the thesis writing process to support brainstorming, structuring chapters, refining explanations, and improving academic tone. It also assisted in simplifying complex sentences for readability.

Sections where AI tools were used:
- **Quadratic**: Results chapter, Data Extraction tables
- **ChatGPT**: Introduction, Related Work, and initial drafts of Methodology

I acknowledge that I am fully responsible for the entire content of my thesis, including the parts generated by AI, and accept accountability for any violations of ethical standards in publications.

PREFACE

This thesis was carried out at Tampere University, within the Faculty of Information Technology and Communication Sciences, during the academic year 2024–2025. The work was conducted under the guidance of Zheying Zhang, whose insights, support and encouragement were instrumental throughout the process.

I am grateful to Tampere University for providing access to digital resources and academic tools that enabled the completion of this study.

Special thanks go to my colleagues, peers, and family for their continuous encouragement throughout this research journey.

Tampere, Finland, 31 July 2025

Syed Babar Ali Rizvi

CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF SYMBOLS AND ABBREVIATIONS

AI	Artificial Intelligence
BERT	Bidirectional Encoder Representations from Transformers
BLEU	Bilingual Evaluation Understudy (evaluation metric)
F1-score	Harmonic mean of precision and recall
GPT	Generative Pre-trained Transformer
IR	Information Retrieval
ISO	International Organization for Standardization
LLM	Large Language Model
MAP	Mean Average Precision
NLP	Natural Language Processing
RAG	Retrieval-Augmented Generation
REQ	Requirement
REQ–Code	Requirement-to-Code Traceability
REQ–Test	Requirement-to-Test Traceability
SMS	Systematic Mapping Study
TLR	Traceability Link Recovery
P	Precision
R	Recall
N	Total number of documents or links
TP	True Positives
FP	False Positives

1. INTRODUCTION

Modern software systems are more complicated, encompassing several interrelated artifacts such as requirements documents, design models, source code, and test cases. Maintaining the relationships between these artifacts — a method known as **requirements traceability** – is critical for successful development and maintenance [1]. Requirements traceability is commonly defined as *"the ability to describe and follow the life of a requirement in both a forwards and backwards direction,"* ensuring that each requirement can be traced from its origins through implementation and beyond [2]. Strong traceability makes it easier for developers and stakeholders to confirm that all requirements are implemented and tested, as well as to comprehend how lower-level artifacts satisfy higher-level needs. Traceability supports critical software engineering activities: it enables change impact analysis (assessing the effects of modifying a requirement), aids bug localization (finding source code related to a defect) and facilitates system maintenance and evolution. For example, if a requirement changes, trace links help discover the related design components, code modules, and test cases that may need to be adjusted. By providing this *"map"* of connections across the project, requirements traceability reduces complexity and improves confidence that the system meets its intended needs [1].

Despite its importance, achieving complete traceability in practice is difficult. Large projects can entail thousands of artifacts and changing requirements, making it impossible to manually build and track all the trace links. As a result, trace links are often incomplete or entirely missing in real-world development projects [3]. The lack of sufficient traceability impairs project visibility and quality: modifications to requirements may mistakenly omit critical code updates, and conflicts between specifications, code, and tests can go undiscovered [1]. Research indicates that lack traceability has tangible harmful consequences. For example, projects with more full trace links had much lower software defect rates. In one study, increased requirements traceability completeness was linked to higher software stability and maintainability [4]. When traceability is weak, it is more difficult to guarantee that all criteria are met and to discover the ripple effects of changes or errors, which can lead to quality issues. These issues underscore why traceability is mandated in safety-critical systems (e.g. by standards like DO-178C and ISO 26262) and widely regarded as a key factor in project success [4]. In conclusion, the increasing

scale and complexity of software, combined with shorter release cycles, has made effective traceability more important than ever, but also more difficult to achieve through purely manual effort [2].

There has long been a significant desire to automate the traceability link recovery process due to the labor-intensive nature of building and maintaining trace links; manually curating links is expensive and time-consuming, and automation promises to alleviate the traceability load [3]. *Traceability Link Recovery (TLR)* refers to the techniques and tools for automatically identifying correspondences (or “trace links”) between artifacts and making those links explicit [1]. In essence, TLR tools attempt to recover the missing links – for example, by finding which pieces of source code implement a particular requirement, or which test cases validate a given design element. TLR has been an active area of research in requirements engineering for many years [4]. Information retrieval (IR) techniques were the focus of early automated TLR approaches, which treated the text of artifacts as documents and ranked possible links based on keyword matching or textual similarity. Although these IR-based methods can rapidly propose plausible links (for example, by comparing code comments to requirement specifications), their precision is sometimes constrained by a lack of semantic comprehension and vocabulary mismatch. Other methods have inferred relationships in particular situations using heuristic principles, such as linguistic patterns or identifier naming conventions [4]. In the past decade, more advanced machine learning and deep learning techniques have also been applied to TLR. Notably, researchers have explored training neural network models on traceability data to capture latent relationships beyond simple keyword overlap [4]. These *deep learning*-based TLR methods have reported improved results in some cases [4], as they can learn to recognize semantic similarities between artifacts (for example, linking a requirement to code even when they do not share exact words). However, despite continuous progress, the state-of-the-art automated approaches still face limitations. Many techniques are specialized to certain artifact types or projects and exhibit highly variable performance across different traceability tasks [1]. Frequently, recovered links' recall and precision fall short of the exacting standards needed for real-world industrial use [1]. In fact, according to practitioner surveys, automated traceability solutions have only been partially implemented in actual projects, partly because of issues with their scalability and accuracy [4]. In summary, automated traceability link recovery is still a challenging task, and there is a known disconnect between the needs of practitioners and what research prototypes can do [4]. The quest for more potent and broadly applicable methods to enhance TLR performance has been sparked by this difficulty.

Recently, the emergence of **Large Language Models (LLMs)** has opened promising new avenues for advancing traceability automation highlighted in Table [1]. LLMs are cutting-edge AI systems – typified by models such as *OpenAI's GPT-3* and its successors – that are trained on massive text corpora and possess an unprecedented ability to understand and generate natural language. These models contain hundreds of millions to billions of parameters, and researchers have found that scaling language models to such sizes yields surprising gains in capability [5]. In fact, when a language model's size exceeds a certain threshold, it not only improves on conventional NLP tasks but also exhibits *emergent behaviors* such as few-shot learning, complex reasoning, and domain adaptation that smaller models struggle with [5]. The launch of ChatGPT, an LLM-based conversational agent, in late 2022 well-illustrated the capabilities of this technology. It can converse, respond to complex queries, and generate coherent writing in a variety of fields, attracting widespread interest from both academia and business [5]. The ability of LLMs to capture language semantics is crucial; they can decipher requirements written in natural language and connect them to documentation or code, even if the two use different languages. This implies that by utilizing a richer knowledge of context, synonyms, and linguistic structure, LLMs may be able to get around some of the drawbacks of earlier tracing techniques. In fact, research has already demonstrated that LLMs perform impressively on a range of software engineering tasks involving natural language, such as source code generation, code summarization, and API recommendation [1]. These successes make a compelling case for exploring LLM-driven approaches in requirements traceability. Researchers have hypothesized that LLMs could enable more *generic and flexible* traceability solutions that work across different artifact types and projects [1] – a significant leap beyond the task-specific algorithms of the past. Initial empirical evidence is encouraging for example, a recent work by Hey et al. applied a LLM with retrieval-augmented generation to recover trace links between textual requirements and reported that this LLM-based approach outperformed several state-of-the-art baselines on benchmark datasets [3]. The accuracy of trace links was increased by using an LLM in conjunction with intelligently retrieving project-specific data, and it was shown that using modern language models to automate some traceability tasks was feasible [3]. However, the study also pointed out that LLMs are not a solution; even while the performance attained was superior to earlier approaches, it was still unable to fully replace manual traceability in every situation [3]. Challenges such as LLM input length limits (which make it hard to feed an entire project's artifacts at once) and the need for project context mean further research is needed to fully realize LLMs' potential in this domain [1]. Nonetheless, the rapid advancements in natural language AI represented by LLM have clearly signaled a new and exciting direction for traceability research.

Table 1. *Comparison of Traditional TLR Methods and LLM-based TLR Approaches*

Approach	Technique	Strength	Limitations
Traditional IR-based TLR	TF-IDF, VSM, lexical matching	Fast, easy to implement; no training required	Vocabulary mismatch; lacks semantic understanding
Heuristic/Rule-based	Pattern matching, naming conventions	Domain-specific precision	Low generalizability; brittle to language variation
ML-based TLR	SVM, Random Forest, neural networks	Learn from labeled examples; better generalization	Needs labeled data; narrow to trained domains
Deep Learning	CNNs, RNNs, embedding similarity	Captures semantics better than IR	Still limited contextual awareness; architecture-specific
LLM-based TLR	GPT, BERT, CodeBERT, RAG	Contextual understanding; few-short/flexible; task-general	Input length limits; can hallucinate; domain adaptation needed

Considering these developments, this thesis focuses on the intersection of **LLMs and requirements traceability.** The goal is to investigate how well LLMs can address the longstanding traceability challenges and to what extent they improve upon traditional methods. By conducting **a systematic mapping study on the use of LLMs for requirements traceability link discovery**, with a particular emphasis on identifying applied techniques, datasets, evaluation strategies, and reported challenges in academic research. This research was conducted at **Tampere University**, under the Faculty of Information Technology and Communication Sciences, and it aspires to contribute both a timely overview and a critical analysis of this emerging field. As software continues to grow in scale and complexity, leveraging smarter automation for traceability becomes increasingly important – the insights from this work will help clarify the state-of-the-art and guide further innovations at the juncture of requirements engineering and artificial intelligence.

1.1 Key Concepts and Definitions:

1.1.1 Large Language Models (LLMs)

Large Language Models are deep neural networks—most often based on the Transformer architecture—that are pre-trained on large-scale textual corpora to perform a variety of language tasks [6]. Notable examples include GPT, BERT, and PaLM. These models have demonstrated strong performance in tasks such as summarization, translation, question answering, and even code generation [7]. Their generalization ability,

especially when combined with techniques like prompting or fine-tuning, allows them to adapt to domain-specific problems such as software engineering.

1.1.2 Requirements Traceability

Requirements traceability refers to the ability to track a requirement's lifecycle and its associations with related artifacts such as design elements, code, and test cases [8]. It is particularly useful for assessing the impact of changes, ensuring test coverage, and verifying that all functional and non-functional requirements have been addressed [9]. Traceability is also mandated by standards like ISO/IEC/IEEE 29148 for critical systems development [10].

1.1.3 Traceability Link Recovery

Traceability Link Recovery is the automated process of identifying and establishing trace links between software artifacts where such links are missing or incomplete [11]. TLR typically involves ranking candidate links based on similar metrics or machine-learned associations [12]. While IR-based methods have been widely used, recent advancements in natural language models present new opportunities for more semantically accurate link prediction.

1.2 Objective and Research Questions

This thesis investigates how LLMs are being applied in the context of Traceability Link Recovery (TLR) in software engineering. The central objective is to systematically map existing research to understand the scope, effectiveness, and limitations of LLM-based approaches for traceability.

The motivation for this study stems from three converging challenges:

- the persistent difficulty of maintaining traceability in large, evolving systems.
- the limited generalizability of traditional TLR methods; and
- the emerging promise of LLMs to bridge semantic gaps across software artifacts.

To guide the mapping process and categorize existing knowledge, the following research questions (RQs) were formulated. Each question reflects a specific knowledge gap or unexplored dimension observed in the literature.

RQ1: What types of traceability link recovery tasks are addressed using LLMs?

Rationale:

Automated traceability is not a monolithic task. It may involve different artifact types (e.g., requirements ↔ test cases, code ↔ design documents), modalities (textual vs. structured), or objectives (e.g., link suggestion vs. link validation). Understanding which subtasks or use cases have been tackled using LLMs helps delineate the current boundaries of research and shows where LLMs are proving effective—or lacking. This also provides insight into task diversity, supporting better generalization across software engineering domains.

RQ2: What LLM-based techniques are applied for TLR (e.g., prompting, fine-tuning, retrieval-augmented generation)?

Rationale:

LLMs can be applied in various ways—zero-shot prompting, few-shot learning, fine-tuning, or through hybrid methods like retrieval-augmented generation (RAG). Each application strategy has trade-offs in terms of data requirements, complexity, interpretability, and performance. This question aims to map the spectrum of implementation strategies and understand how researchers are leveraging LLM capabilities to suit different traceability tasks. It also helps clarify the balance between off-the-shelf use and task-specific adaptation.

RQ3: What datasets, tools, and evaluation strategies are used to assess LLM-based traceability link recovery?

Rationale:

Evaluation is critical to both scientific progress and industrial trust. This question explores what benchmarks, corpora, or toolkits are being used to test LLM-based TLR solutions. It also examines how success is measured—e.g., precision, recall, F1-score, human validation—and whether these metrics are consistent or comparable across studies. The rationale is to identify possible gaps in reproducibility, standardization, and availability of datasets, which are recurring issues in software traceability research.

RQ4: What challenges or limitations are reported in applying LLMs for TLR, especially in industrial contexts?

Rationale:

While early results with LLMs are promising, it is important to understand the practical barriers to wider adoption, especially in industrial environments. This question aims to uncover commonly reported issues such as hallucinations, domain adaptation needs, cost constraints, input length limitations, or explainability concerns. It also looks for qualitative observations, such as how engineers perceive LLM-generated trace links. This

knowledge is crucial for bridging the gap between academic prototypes and real-world deployment.

1.3 Scope and Delimitations

The scope of this thesis is defined by the inclusion and exclusion criteria used during the systematic mapping process. Only **peer-reviewed literature** published between 2015 and 2025 is considered. The selected works must report empirical results or propose methods related specifically to the application of large language models in traceability link recovery tasks. General-purpose NLP papers, non-traceability studies, vision papers, and gray literature are excluded.

The review draws exclusively from the **IEEE Xplore**, **ACM Digital Library**, **Scopus and Web of Science** databases. These databases were chosen for their breadth and relevance in software engineering and computer science. Furthermore, only studies published in English and available in full text were included. Studies focusing on other aspects of requirements, engineering or traceability (e.g., visualization, tool integration) are discussed only if they directly intersect with LLM usage.

2. RELATED WORK

2.1 Early Information Retrieval-Based Approaches to Traceability

Early approaches to traceability link recovery (TLR) treated the problem as one of document similarity and information retrieval (IR). Classic methods such as the vector space model (VSM) and latent semantic indexing (LSI) were applied to measure textual similarity between artifacts (e.g. requirements and code) and infer links [13]. For example, Antoniol *et al.* [14] used simple VSM cosine similarity to recover links between code and documentation, while Marcus and Maletic introduced LSI to capture latent semantic relationships in documentation-to-code tracing [15]. These IR-based techniques established a baseline for automated trace link generation without requiring training data. However, they often suffered from vocabulary mismatch and the *semantic gap* – i.e. differing terminologies between artifacts – which limited their accuracy [16]. Researchers attempted to bridge this gap by augmenting IR methods with semantic information. Some approaches expanded the text corpora with related documentation [13] or incorporated domain knowledge (e.g. via WordNet or semantic relatedness measures) to detect links that pure keyword matching would miss [13]. Others integrated additional project data; for instance, mining version control and bug databases to validate candidate links (as in *Trustrace* by Ali *et al.*) [17]. Despite incremental improvements, these traditional methods often plateaued in performance and struggled to achieve the high precision and recall needed for practical use. In fact, studies have noted that even state-of-the-art traceability techniques frequently fall short of the accuracy required in real-world settings [1]. This motivated a shift towards learning-based solutions that could better model the complex semantics between artifacts.

2.2 Machine Learning and Early Deep Learning for TLR

To overcome the limitations of purely IR-based techniques, researchers explored machine learning models to learn the linking patterns from data. Early work formulated TLR as a binary classification task given a pair of artifacts, predicting whether a trace link exists. Mills *et al.* [18] pioneered this direction by training classifiers on existing trace links to automate link maintenance. In their 2018 study, a machine learning model was trained to verify links, and later work applied active learning to reduce the amount of

labeled data needed [18]. Such classification-based methods demonstrated that incorporating learned patterns could outperform static IR scores, especially when moderate training data was available.

The advent of deep learning brought further advancements. Ruan *et al.* [19] proposed **DeepLink**, among the first deep neural approaches for traceability, focusing on issue–commit link recovery. DeepLink learned joint representations of issue reports and commit messages using word embeddings and **recurrent neural networks** (**RNNs**) to capture their semantic relationships [19]. Around the same time, Guo *et al.* [12] introduced **Traditional Neural Networks (TNN)** for general traceability tasks, which also leveraged word embeddings with an RNN to address vocabulary mismatch between artifacts [12]. By encoding textual artifacts into vector representations and then learning to predict links, these deep learning models could infer links that IR methods missed (for example, linking synonyms or conceptually related terms). Empirical results showed that deep models often achieved higher recall and precision than classic IR, confirming the effectiveness of learning latent features [13].

However, a major challenge for early deep TLR models was the scarcity of labelled training data. Supervised deep networks require large datasets, yet in practice only limited trace links are available (since creating them manually is labour-intensive) [13]. This data sparsity caused some deep models to underperform or overfit. For instance, a simple RNN-based model trained on sparse project data could barely surpass IR in accuracy [16]. Researchers addressed this problem through techniques like data augmentation and semi-supervised learning. Mills *et al.* [18] used active learning to iteratively label the most informative candidate links, thereby improving the classifier with minimal human effort. Xiao *et al.* [13] presented **TRACEFUN**, a framework to exploit unlabelled artifacts in addition to the few labelled links. TRACEFUN automatically generated new training links by identifying semantically similar artifact pairs via contrastive learning and VSM, then assuming those with shared neighbours should be linked [13]. By adding these pseudo-labelled links to the training set, they boosted deep model performance substantially – improving F1-scores by up to 21% for a BERT-based model and even 10× for an RNN model on one dataset [13]. These efforts underscored the importance of data in TLR and paved the way for leveraging large pre-trained models that could transfer knowledge from other sources.

2.3 Pre-trained Transformer Models for Traceability

A significant breakthrough in traceability came with the adoption of large pre-trained language models, especially Transformer-based models like BERT. Instead of training a

deep model from scratch on limited trace data, researchers fine-tuned models that had been pre-trained on massive corpora of text (and code). **Traceability Transformed (Trace BERT)** by Lin *et al.* [16] exemplified this approach. Trace BERT harnessed BERT's rich contextual understanding by using a three-step training process: first pre-training on a general software corpus, then transferring knowledge from a related task (such as code search), and finally fine-tuning on the traceability data. This strategy dramatically improved link recovery accuracy despite the small target dataset. In an evaluation on open-source projects, the best Trace BERT model significantly outperformed classical IR baselines – achieving over a 60% improvement in Mean Average Precision compared to a VSM approach [16]. All tested BERT-based configurations (single BERT and Siamese BERT architectures) surpassed the traditional methods, indicating the power of pre-trained language representations to bridge the semantic gap. Notably, even when an earlier RNN approach failed due to insufficient training data, the BERT-based model still performed robustly by virtue of its prior knowledge. These results marked a new state-of-the-art, where learning-based TLR could finally outperform IR by a large margin on real-world traceability tasks.

Following Trace BERT, other transformer models and embeddings were applied in traceability. CodeBERT, a variant pre-trained on source code, was used to compute embeddings for artifacts and showed better recall than plain text embeddings on code-related links [20]. Researchers also explored different model architectures: Lin *et al.* [16] reported that a cross-encoder (Single-BERT) setup yielded the most accurate links, while a Siamese (bi-encoder) BERT was nearly as good and much faster. The choice of architecture involves a trade-off between accuracy and efficiency, but in all cases the leverage of pre-trained transformers was key. By 2021-2022, fine-tuned transformers became the new baseline for automated traceability link recovery, largely replacing earlier pure IR or simple deep learning methods in research literature. However, fine-tuning requires careful training and still demands labelled examples for each specific traceability task. This limitation motivated researchers to investigate whether *off-the-shelf* large language models could perform traceability through prompting, without task-specific training.

2.4 Large Language Models and Retrieval-Augmented Techniques

The rapid progress in **large language models (LLMs)** like GPT-3 and GPT-4 opened new avenues for traceability. These models, with billions of parameters, possess an advanced understanding of natural language (and even programming language) learned from vast data. The question arose: can an LLM, given a suitable prompt, recover trace

links between software artifacts? Early explorations indicate that LLMs indeed hold promise in this domain. In a recent empirical study, the best LLM achieved F1-scores around 80% on documentation-to-code traceability tasks, substantially higher than classical methods like TF-IDF, BM25, or even a CodeBERT embedding approach on the same data [20]. This demonstrates that with zero-shot or few-shot prompting, generative LLMs can utilize their semantic knowledge to bridge the gap between requirements or docs and low-level code details [20]. An additional benefit is that LLMs can produce human-readable explanations for why a link is suggested. Unlike earlier black-box models, an LLM might output a rationale (e.g. *"this code method implements the functionality described in that requirement"*). In evaluations, 40–70% of these LLM-generated explanations have been rated fully correct, and nearly all are at least partially correct [20], reflecting a new level of transparency in automated traceability.

Despite their strengths, applying LLMs to TLR is not straightforward. One challenge is **context limitation**: an LLM cannot ingest an entire large codebase or document set at once due to input length constraints [1]. Key project-specific details might be outside the prompt window. To address this, researchers have turned to **retrieval-augmented generation (RAG)** strategies. In RAG, an IR step first retrieves a subset of relevant artifacts for a given source, and then the LLM is prompted with those artifacts to determine links. This hybrid approach provides the model with focused context, essentially **combining classical IR strengths with LLM reasoning**. Fuchβ *et al.* [1] developed a framework called **LiSSA (Linking Software System Artifacts)** that generalizes traceability via RAG. LiSSA uses a search engine to fetch candidate targets for a given source artifact, then asks the LLM (with chain-of-thought prompting) to decide which candidates truly trace to it [3][1]. By doing so, LiSSA was able to tackle multiple traceability scenarios (requirements-to-code, documentation-to-code, and even architecture model links) with the same approach. Empirical results are encouraging: the RAG-based method significantly outperformed prior state-of-the-art methods on the code-related tasks, nearly doubling accuracy in some cases. Similarly, for linking textual requirements to each other, chain-of-thought prompting with an LLM improved the link recovery performance and even open-source LLMs performed on par with proprietary models in that setting [3].

Another notable example is the **EasyLink** approach by Huang *et al.* [21] targeting issue–commit traceability. EasyLink indexes software commits in a vector database and retrieves a shortlist of candidates for a given issue description, which an LLM then re-ranks and evaluates. This LLM-assisted retrieval proved remarkably effective: on a realistic large-scale dataset, EasyLink achieved about 75.9% Precision@1, more than four times

higher than the previous best deep learning method on the same task. Notably, in scenarios with many distracting candidates, a simple VSM baseline had outperformed a deep learning model, highlighting robustness issues in the learned model. The LLM, with its stronger semantic reasoning, was able to discern the correct links among many plausible but wrong candidates, thus dramatically boosting precision [21]. These findings underscore that LLMs can complement traditional retrieval: the IR component narrows down the search space, and the LLM provides a sophisticated analysis to pick the true link.

While large language models are pushing the frontier of automated traceability, researchers also caution that they are **not a silver bullet**. Even the best LLM-based approaches have yet to consistently reach the near-perfect accuracy needed for fully autonomous trace link generation in practice [3]. For instance, Hey *et al.* [3] report that despite improvements, their LLM + RAG method still might miss links or introduce false links, meaning human oversight or validation is still necessary in critical projects. Moreover, LLMs sometimes struggle with *multi-step trace chains* (linking artifacts through intermediate steps) and with providing completely correct explanations for complex relationships [20]. There are also practical concerns: running large models can be computationally expensive, and using proprietary LLM APIs raises issues of cost and data privacy. Recent studies have shown that smaller open-source LLMs, when properly prompted and augmented with retrieval, can perform comparably to giant proprietary models [3]. This opens avenues for more accessible solutions but also highlights the need for further research in fine-tuning LLMs for traceability tasks and handling domain-specific knowledge.

In summary, the field has evolved from simple textual similarity methods to sophisticated AI-driven techniques. Each generation of approaches – IR, classical ML, deep learning, pre-trained transformers, and now prompt-based LLMs – has brought improvements in capturing the semantic links between software artifacts. The latest LLM-based methods achieve unprecedented accuracy and the ability to explain links, making traceability tools more powerful and user-friendly than ever. However, they also introduce new challenges (context handling, reliability, cost) that researchers are actively exploring. **The emergence of large language models is reshaping traceability research**, and our work is positioned in this context: reviewing and synthesizing how these advanced models have been applied to traceability link recovery, what benefits they offer, and what gaps remain. By grounding the discussion in peer-reviewed findings, we ensure a balanced view – acknowledging both the breakthroughs and the limitations – to inform future developments in automated software traceability.

3. METHODOLOGY

3.1 Research Approach

This study follows the principles of a Systematic Mapping Study (SMS), adapted from the guidelines proposed by Petersen et al. [22] for software engineering research. The objective of the SMS is to provide a structured overview of how large language models (LLMs) have been applied to requirements traceability link recovery (TLR), including the types of models used, evaluation practices, and reported challenges. SMS as shown in Fig [1] is a structured approach for surveying, categorizing, and visualizing the breadth of existing research in each area. Unlike systematic literature reviews (SLRs), which aim to answer narrowly focused questions through critical appraisal and synthesis, SMSs are broader in scope and more exploratory in nature. They are particularly well suited to identifying research trends, gaps, and the distribution of studies across various facets of a topic [22][23].

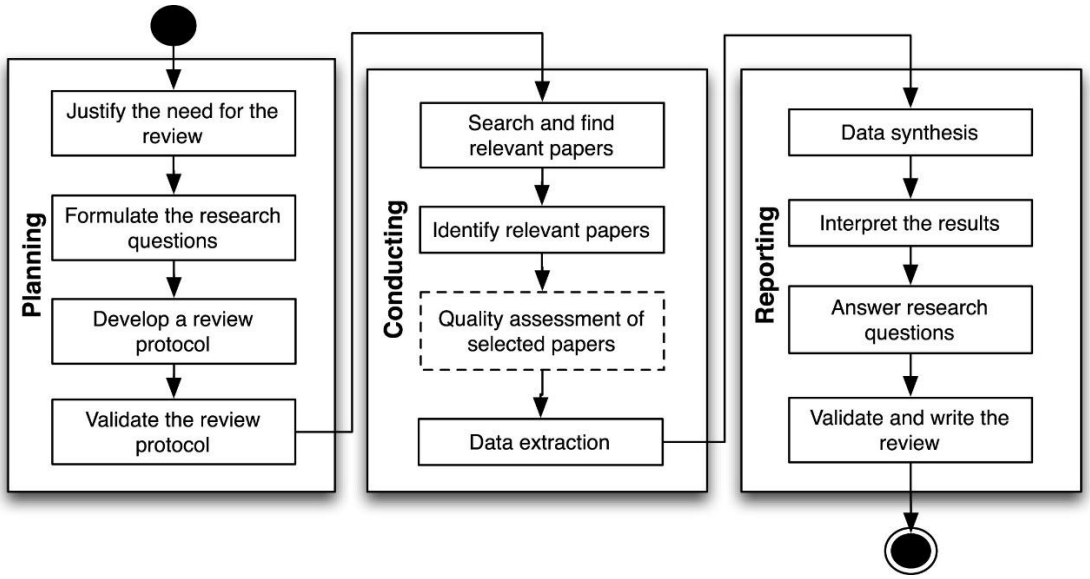

Figure 1. *Systematic mapping study process by Hannousse et al [25].*

The choice of SMS is justified by the exploratory nature of the research questions, which aim to understand how Large Language Models (LLMs) are applied to traceability link discovery tasks across multiple research domains. The questions focus not on evaluating the performance of a specific technique, but on mapping the landscape: which models have been used, in what contexts, with what data, and for which types of traceability problems. An SMS is suitable for this type of objective because it facilitates the development of classification schemes, the quantification of research activity, and the visualization of coverage across different dimensions.

SMS has become an established method in empirical software engineering, particularly in fast-evolving research domains such as AI-assisted development. Its structured process includes clearly defined stages: formulating research questions, constructing a search strategy, applying inclusion/exclusion criteria, screening and selecting primary studies, extracting and coding data, and classifying results [22][24].

The structure of this mapping study adheres to established guidelines and procedures proposed by Petersen et al. [22] and Kitchenham and Charters [23], which are widely accepted in software engineering research. These frameworks ensure replicability, transparency, and rigor, which are essential for academic credibility and future extensions of the study.

This approach enables the thesis to systematically investigate how LLMs have been adopted for traceability link discovery, what categories of techniques and tools have been applied, and what limitations or challenges are commonly reported in peer-reviewed research.

3.2 SMS Protocol Overview

To address the research questions in a rigorous and transparent manner, this thesis followed a Systematic Mapping Study (SMS) protocol adapted from established guidelines in software engineering [22][23]. The protocol was designed to ensure repeatability, minimize bias, and align with best practices in evidence-based research.

The mapping protocol includes clearly defined steps: goal formulation, search strategy development, inclusion and exclusion criteria, source selection, and quality assurance. A high-level summary is provided in Table [2], and each component is described briefly below.

Table 2. *Overview of the Systematic Mapping Study Protocol*

Step	Description
Goal	To systematically map peer-reviewed literature on how large language models (LLMs) support traceability link recovery in software engineering, with a focus on techniques, datasets, tool support, evaluation strategies, and industrial applicability.
Search Strategy	Search string using keywords: (requirement OR issue* OR "user stor*" OR "use case") AND (trace* OR tracing) AND (llm OR "large language model" OR "foundation model" OR "generative AI" OR "gen-AI" OR bert OR gpt OR gemini OR claude OR llama)
Inclusion/Exclusion Criteria	Include peer-reviewed literature addressing LLM-based traceability tasks; Exclude

	gray literature, non-English sources, and unrelated domains.
Quality Assessment	Peer-review status is used as a primary quality filter.

3.2.1 Protocol Phases

Search Strategy:

A Boolean search string was constructed using keyword variants relevant to requirements engineering, traceability, and large language models. The string includes terms such as "requirement" OR "issue" OR "user story" combined with "traceability" and model-related terms like "LLM", "GPT", or "BERT".

Inclusion/Exclusion Criteria:

Only peer-reviewed publications were included to ensure quality. Articles must describe the use of LLMs for traceability-related tasks. Vision papers, non-English papers, unpublished theses, and blog posts were excluded.

Quality Assessment:

As suggested by Petersen et al. [22] publication venue and peer-review status were used as proxy indicators for research quality. Additional quality metrics, such as completeness of method descriptions and clarity of evaluation, were noted during data extraction but not used as filters at the selection stage.

3.3 Research Questions

This systematic mapping study is guided by four research questions that collectively aim to explore how Large Language Models (LLMs) are being applied to the task of traceability link recovery in software engineering. These questions are designed to investigate the technical scope, empirical foundations, tool support, and reported limitations of LLM-based approaches.

The research questions are broad in nature, consistent with the aims of a mapping study, and are intended to support classification rather than synthesis. They were formulated based on a review of related work and refined through pilot screenings to align with trends and gaps observed in the current literature [22].

RQ1: What types of traceability link recovery tasks are addressed using LLMs?

This question investigates the practical scope of traceability tasks explored in the literature. It aims to identify which types of artifact pairs (e.g., requirements-to-code, requirements-to-test) have been studied, and whether the LLMs are applied to generate trace

links, validate existing ones, or assist in understanding traceability relationships. This helps clarify what tasks LLMs are currently supporting in real-world or experimental settings.

RQ2: What LLM-based techniques are used for traceability link recovery (e.g., prompt engineering, fine-tuning, RAG)?

This question focuses on the technical strategies researchers are employing to apply LLMs in traceability contexts. It distinguishes between prompting-based methods, fine-tuned models, and retrieval-augmented generation (RAG), among others. It also aims to identify which families of models (e.g., GPT, BERT, Gemini, Claude, LLaMA) are used. Understanding these techniques is essential for assessing the flexibility, complexity, and adaptability of LLM-based solutions.

RQ3: What datasets, tools, and evaluation strategies are used to assess LLM-based traceability link recovery?

This question targets the empirical foundations of LLM-based TLR research. It examines the types of datasets used (e.g., open-source, industrial, synthetic), the evaluation metrics adopted (e.g., precision, recall, F1-score), and whether human validation is part of the methodology. The aim is to assess the reproducibility, rigor, and comparability of the studies.

RQ4: What challenges or limitations are reported in applying LLMs for traceability link recovery, especially in industrial contexts?

This question explores the reported barriers to applying LLMs in traceability settings, particularly in practical or industrial deployments. It considers issues such as scalability, model hallucination, domain adaptation, explainability, and system integration. Understanding these challenges is critical for identifying research gaps and informing the development of more robust LLM-based traceability solutions.

These research questions serve as the foundation for the classification scheme, search string formulation, inclusion/exclusion decisions, and overall analysis reported in later sections of this thesis.

3.4 Search Strategy

The goal of the search strategy was to retrieve a comprehensive and relevant set of peer-reviewed publications that discuss the application of Large Language Models (LLMs) to requirements traceability link discovery. This process followed best practices

outlined in the SMS methodology literature [22][26], including iterative refinement and alignment with the research questions defined in Section 3.3.

3.4.1 Database Selection

To ensure disciplinary breadth and indexing quality, three widely used and peer-reviewed scholarly databases were selected:

IEEE Xplore: for software engineering, systems, and AI research

ACM Digital Library: for computing and software conference publications

Scopus: for multidisciplinary, indexed journal and conference articles

These databases are widely used in empirical software engineering studies and provide robust filtering for peer-reviewed content [27].

3.4.2 Search String Formulation

The search string was constructed to combine terms across three conceptual categories:

(1) software artifacts (requirement-level expressions),

(2) traceability tasks, and

(3) LLM or generative AI terminology.

The final search string was as follows:

(requirement OR issue* OR "user stor*" OR "use case")
AND (trace* OR tracing)
AND (llm OR "large language model" OR "foundation model" OR "generative AI" OR "gen-AI" OR bert OR gpt OR gemini OR claude OR llama)

This formulation used wildcards (e.g., issue*, trace*) to capture plural and variant forms, as well as synonyms and emerging terms such as "foundation model" and "gen-AI". It was designed to account for both general and model-specific mentions (e.g., bert, gpt, gemini, claude, llama) in line with the evolving landscape of LLM research.

Each database required minor syntax adaptation (e.g., quotes, field scoping). For example, in Scopus, the string was applied using the TITLE-ABS-KEY field scope.

3.4.3 Publication Time Frame and Language

The search included publications from January 2015 to June 2025, capturing both early neural approaches to traceability and the more recent wave of LLM-based methods following the release of models like BERT (2018) and GPT-3 (2020). Only English-language, peer-reviewed publications were considered. Preprints, non-reviewed blog posts, and gray literature were excluded to ensure methodological rigor [26].

3.4.4 Search Execution and Validation

A pilot test of the string was run in each database to evaluate its precision and recall. Based on the pilot results:

Synonyms such as "user stor*" and "use case" were retained to expand scope

Terms like "generative AI" and "foundation model" helped capture newer terminology

False positives (e.g., unrelated uses of “issue”) were reviewed and managed during screening

The final search results were exported, and duplicates were removed as attached in the SMS log sheet (Appendix A)

3.5 Inclusion and Exclusion Criteria

The selection of studies in this mapping followed a rigorous screening process, guided by well-defined inclusion and exclusion criteria. These criteria were informed by established practices in systematic mapping studies [22][26] and further refined based on pilot screenings. The goal was to include only empirical, peer-reviewed studies that specifically explore the use of Large Language Models (LLMs) in the context of requirements traceability.

3.5.1 Inclusion Criteria

A paper was included if it satisfied at least one of the following criteria:

- It reports an empirical study, such as a case study, controlled experiment, user study, interview, survey, benchmarking exercise, or evaluation using real-world datasets.
- It involves real-world use or industrial evaluation of LLM-based traceability methods.
- It addresses the use of LLMs (e.g., GPT, Claude, Gemini, or open-source LLMs like BERT, LLaMA) specifically for requirements traceability tasks.

- It explores LLM-based techniques such as prompt engineering, fine-tuning, or retrieval-augmented generation (RAG) in the context of trace link recovery.
- It proposes a tool, method, or framework that supports traceability within requirements engineering using LLMs.

These criteria were selected to capture both theoretical contributions with practical implications and applied research involving actual system artifacts and LLM implementations.

3.5.2 Exclusion Criteria

A paper was excluded if it met any of the following:

- It was not written in English.
- It was not published between 2015 and 2025.
- It offered only theoretical observations without empirical validation.
- It was a comparison study between tools unrelated to LLMs or RE traceability.
- It was a duplicate publication (e.g., earlier version of a journal article).
- It was a research plan, roadmap, or vision paper lacking technical implementation.
- It used LLM terms (e.g., “traceability,” “GPT”) out of context or for unrelated domains.
- It was a non-peer-reviewed source (e.g., blog post, preprint without review, thesis).

These exclusion rules ensured that the included studies provided substantial and verifiable contributions to the field of LLM-assisted requirements traceability.

3.5.3 Rationale and Pilot Application

The criteria were initially tested through a pilot screening of a subset of papers to verify their clarity and utility. This exercise helped confirm that the selected criteria filtered out low-quality or off-topic studies while retaining relevant, high-impact work. The rules were applied consistently during title, abstract, and full-text screening, and any ambiguous cases were resolved through discussion among reviewers.

3.6 Study Selection and Screening Process

A structured and rigorous study selection process was followed to ensure that only the most relevant and high-quality research articles were included in this mapping. The process was conducted in multiple stages and adhered to established practices for systematic mapping studies [22][26]. It involved database selection, record retrieval, language filtering, duplicate removal, and application of the predefined inclusion and exclusion criteria described in Section 3.5.

3.6.1 Initial Retrieval and Deduplication

The initial search yielded a total of 312 records across three major digital libraries:

IEEE Xplore: 83 records

Scopus: 228 records

ACM Digital Library: 1 record

Following retrieval, 13 non-English papers were excluded. Subsequently, 41 duplicate papers were removed by comparing article titles, DOIs, and metadata across Scopus and IEEE Xplore. Deduplication was manually validated to prevent loss of unique studies.

3.6.2 Screening and Eligibility Assessment

The remaining articles were subjected to a three-stage screening process:

Title Screening: Articles with irrelevant titles or clearly unrelated domains were discarded (e.g., non-software or unrelated uses of “traceability” or “language model”).

Abstract Screening: The abstracts of the remaining articles were reviewed to assess initial relevance to the use of LLMs in traceability link recovery.

Full-Text Screening: Studies passing the first two filters were reviewed in full to ensure they met at least one inclusion criterion and none of the exclusion criteria.

This step confirmed the presence of empirical evidence, use of LLM-based methods, and relevance to software traceability tasks.

3.6.3 Final Inclusion Result

After all filtering and screening stages, the final set of included papers *(see Fig [2])* consisted of:

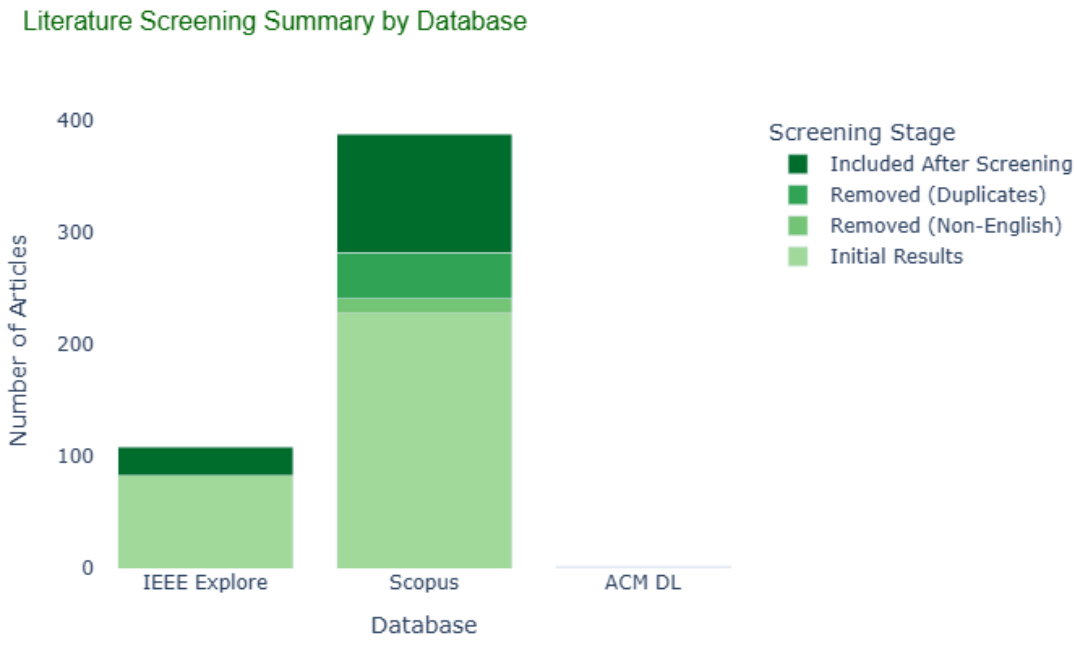

Figure 2. *Literature Screening Summary*

IEEE Xplore: 25 papers

Scopus: 106 papers

ACM Digital Library: 1 paper

This yielded a total of 132 peer-reviewed studies deemed eligible for inclusion and classification as shown in Table [3].

Table 3. *Literature Screening Summary*

Database	Initial Results	Removed (Non-English)	Removed (Duplicates)	Included After Screening
IEEE Explore	83	0	0	25
Scopus	228	13	41	106
ACM DL	1	0	0	1
Total	312	13	41	132

3.7 Data Extraction and Classification Scheme

Following the screening process outlined in Section 3.6, this phase involved the systematic extraction and classification of information from each included study. The goal was

to organize the evidence in a structured and analyzable format, consistent with the standards for systematic mapping studies in software engineering [22][28].

A total of 132 unique peer-reviewed publications were included for data extraction:

106 from Scopus

25 from IEEE Xplore

1 from ACM Digital Library

These studies were coded across multiple dimensions aligned with the research questions defined in Section 3.3.

3.7.1 Data Extraction Procedure

Each included study was reviewed in full text, and a structured Excel template was used to extract and record metadata and classification codes. The extracted fields included both descriptive metadata and content-related properties, such as:

- Bibliographic information (title, authors, venue, year, DOI)
- Abstract and summary
- Artifact types involved (requirements, code, tests, etc.)
- Type of traceability task (generation, validation, explanation)
- Use and type of Large Language Model (LLM)
- Technique (e.g., prompting, fine-tuning, RAG)
- Dataset type and source (e.g., public, industrial)
- Evaluation metrics (e.g., F1-score, human review)
- Reported limitations or practical challenges

Cases of ambiguity—such as inferred LLM usage or indirectly reported evaluation methods—were noted with a rationale beside the classification. This process aligns with the transparency guidelines recommended by Petersen et al. [22].

3.7.2 Classification Framework

The hierarchical visualization Fig [3] shows the relationships between research questions and their associated classification facets. RQ3 appears as the central research question with the most dimensions, reflecting the importance of evaluation in this research field.

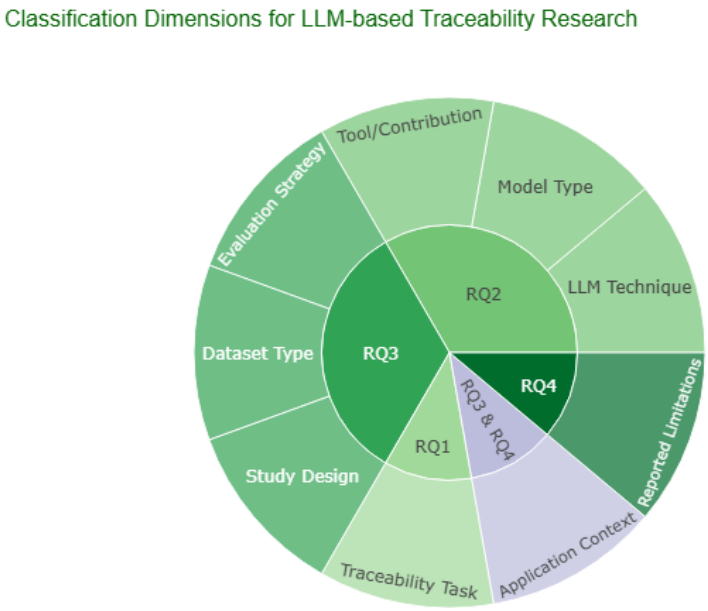

Figure 3. *Classification Dimensions*

Multi-label classification was used where applicable as shown in Table [4]. For instance, if a study used both fine-tuning and prompting, or involved both REQ–Code and REQ–Test links, both categories were recorded.

Table 4. *Categorization by following dimensions*

Facet	**Example Categores**	**Aligned Research Question**
Traceability Task	REQ–Code, REQ–Test, REQ–REQ, REQ–Design	RQ1
LLM Technique	Prompting, Fine-tuning, Retrieval-Augmented Generation (RAG)	RQ2
Model Type	GPT-3/4, BERT, CodeBERT, Claude, Gemini, LlaMA	RQ2
Tool or Contribution	Framework, Method, Plugin, API Wrapper	RQ2
Evaluation Strategy	F1-score, Precision, Recall, BLEU, Human Review	RQ3
Dataset Type	Open-source, Industrial, Synthetic	RQ3
Study Design	Case Study, Controlled Experiment, User Study, Benchmarking	RQ3
Application Context	Academic, Industry, Simulated Environment	RQ3 & RQ4
Reported Limitations	Input token limit, Hallucination, Generalization gap, Cost	RQ4

3.7.3 Justification and Refinement

The classification scheme was developed through an iterative process, combining deductive categories from prior studies (e.g., [22][28]) with inductive adjustments observed during initial paper coding. A pilot ran on 10 randomly selected studies that helped refine facet definitions and resolve classification ambiguity.

Following Budgen et al. [28], the scheme was deliberately broad yet structured, supporting both macro-level trends and fine-grained distinctions across methods and evaluation approaches.

3.8 Validity Considerations

Ensuring the validity of a systematic mapping study requires addressing potential biases and limitations at each stage of the process. In line with established guidelines for evidence-based software engineering [26], this section outlines the primary **threats to validity** encountered in this study and the **strategies** employed to mitigate them.

3.8.1 Selection Bias

Selection bias may occur if the search strategy fails to capture relevant studies or overrepresents certain types of research. To reduce this threat, the study employed:

- **Multiple digital libraries** (Scopus, IEEE Xplore, ACM DL) to broaden coverage;
- A carefully constructed and **piloted search string**, refined based on preliminary runs;
- Inclusion of **synonyms and emerging terminology** (e.g., "gen-AI", "foundation model") to reflect evolving LLM vocabulary.

Additionally, duplicate filtering and non-English exclusions were handled systematically to maintain consistency across sources.

3.8.2 Interpretation Bias

Interpretation bias may arise during classification and data extraction, especially when publication details are ambiguous or incomplete. To address this, the following steps were taken:

- A **structured data extraction form** was used to ensure consistency;
- Each paper was reviewed by at least one researcher, and ambiguous cases were **annotated with rationale**;

- The classification scheme was developed **iteratively**, incorporating adjustments from pilot trials and feedback during coding;
- Multi-label classification was used where studies spanned multiple categories, reducing forced categorization.

These strategies helped improve reliability and reduce subjective misclassification.

3.8.3 Reporting and Scope Limitations

As with all mapping studies, the study's findings reflect **published and peer-reviewed** work only; gray literature and non-English publications were excluded. While this maintains academic quality, it may omit insights from emerging, non-traditional sources.

Additionally, some papers lacked complete reporting (e.g., on dataset names or evaluation methods), which may affect the depth of classification for certain dimensions. When such cases occurred, conservative judgment was applied, and a "not reported" tag was used to preserve transparency.

3.8.4 Construct Validity

Construct validity refers to the degree to which the study accurately captures what it intends to measure—in this case, peer-reviewed empirical research on the use of LLMs for traceability link recovery.

Threats:

- Incomplete or imprecise definition of constructs such as "LLM," "traceability link recovery," or "empirical study" may result in irrelevant inclusions or missed studies.
- Ambiguity in paper terminology (e.g., some authors refer to LLMs indirectly, or use "traceability" in non-software contexts).

Mitigation Strategies:

- A detailed protocol was defined to operationalize key concepts.
- Inclusion/exclusion criteria were grounded in well-established definitions from the literature.
- Pilot runs of the search string ensured that the queries retrieved representative and relevant papers.

3.8.5 Internal Validity

Internal validity relates to the extent to which the findings are attributable to the study process rather than bias or researcher error.

Threats:

- Potential bias in judgment during inclusion/exclusion or during classification of studies.
- Inconsistency in interpreting study details across different reviewers.

Mitigation Strategies:

- A multi-phase screening process was used (title, abstract, full text).
- A subset of studies was used to **pilot and refine the classification scheme** before full coding.
- When feasible, dual review was applied; otherwise, a primary review with rationale annotation was recorded.
- A transparent classification structure with multi-label support minimized forced categorization.

3.8.6 External Validity

External validity concerns the generalizability of the findings beyond the included sample.

Threats:

- Restriction to English-language, peer-reviewed literature may exclude relevant industrial or non-traditional work (e.g., whitepapers, technical reports).
- Inclusion only of published research from three databases (Scopus, IEEE Xplore, ACM) may overlook findings indexed elsewhere.

Mitigation Strategies:

- The use of three comprehensive digital libraries helped ensure broad coverage across journals and conferences in software engineering and AI.
- The decision to exclude gray literature was intentional to maintain academic quality and replicability.
- Industrial relevance was captured explicitly in the classification (e.g., industrial datasets, industry-authored studies), helping identify the real-world applicability of the findings.

This chapter has described the research design and methodology of the systematic mapping study. It detailed the formulation of research questions, database search strategy, inclusion and exclusion criteria, study screening process, data extraction, classification, and validity considerations.

The study followed rigorous practices recommended for evidence-based software engineering [22][26], resulting in the inclusion of 125 peer-reviewed articles published between 2015 and 2025. These articles were systematically classified across multiple facets, including traceability tasks, LLM techniques, evaluation strategies, datasets, and reported challenges.

The next chapter presents the results of the mapping and provides a structured synthesis of how LLMs are currently used for requirements traceability link recovery.

4. RESULTS

4.1 Publication Trends and Study Overview

The **publication timeline** highlighted in Fig [4] reveals that research on applying LLMs to traceability link recovery is very recent and rapidly growing. *No papers were identified prior to 2020*, and the first relevant studies only appeared around 2021–2022. Thereafter, the annual publication count rose sharply. In 2022, only a **handful of papers** (low single digits) were published, but this number grew in 2023 and **spiked dramatically in 2024**. In fact, more papers were published in **2024 alone than in all previous years combined**, reflecting the surge of interest following the emergence of powerful LLMs (e.g. GPT-3/4). This trend continues in 2025 – by mid-2025 the volume of publications is on track to **match or exceed the 2024 output**. Overall, over **85%** of the studies in our dataset were published in **2023 or later**, indicating that LLM-based traceability is a very recent focus in the literature.

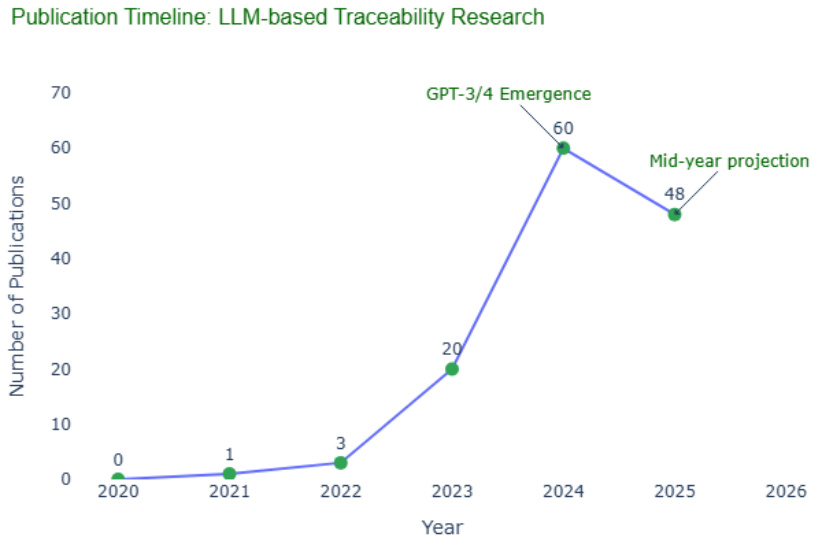

Figure 4. *Evolution of Studies Per Year – 2015 - 2025.*

In terms of **study types**, the vast majority of papers involve some form of empirical evaluation reflected in clustering analysis shown in Fig [5]. Approximately **nine out of ten studies** (90%) report *empirical studies* – for example, experiments, case studies, or user evaluations – to assess the proposed traceability approaches. Only a small fraction (roughly 10%) are *non-empirical* contributions such as conceptual frameworks, vision papers, or literature reviews without a new experiment. Notably, our dataset includes a **few secondary studies** (e.g. a recent systematic review of traceability techniques), but

these are rare. Many papers introduce a new **method or tool** (e.g. novel LLM-based link recovery techniques), typically accompanied by an initial evaluation, rather than purely theoretical work.

We also examined the degree of **industrial involvement** in these studies. About **one-third** of the publications (30–40%) had some *industrial applicability or participation* – for instance, studies conducted in collaboration with industry partners, evaluations on industrial datasets, or authors affiliated with industry. The majority, however, were carried out in academic or lab settings without direct industry participation. This suggests that while industrial interest in LLM-driven traceability is emerging, most research is still in an exploratory or proof-of-concept stage within academia.

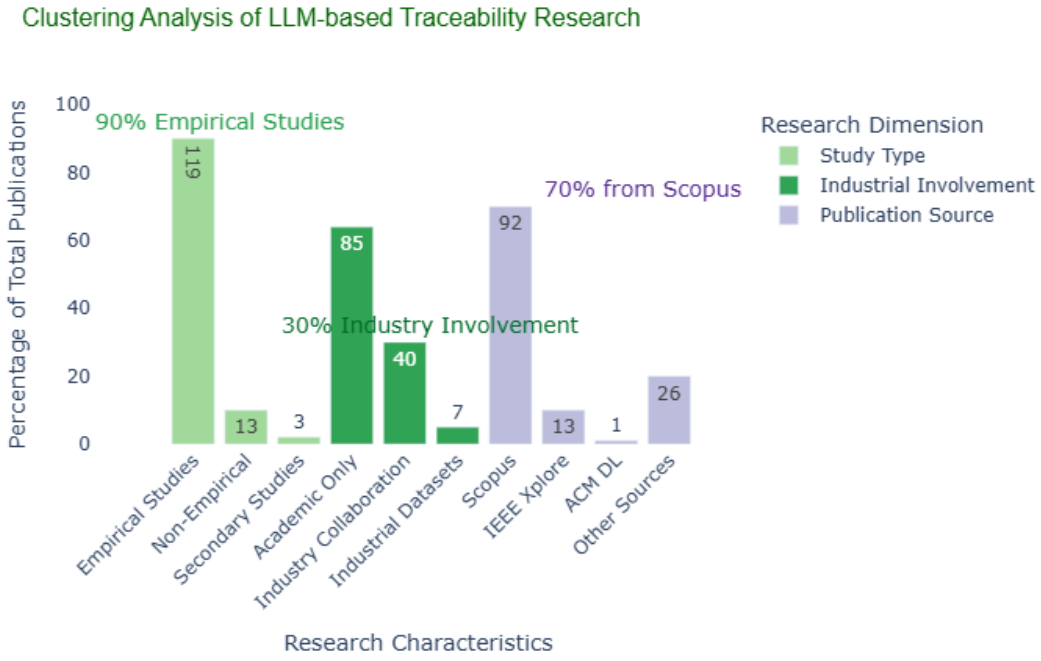

Figure 5. *Clustering Analysis of LLM-based Traceability*

Table [5] summarizes the **distribution of the 132 papers by source**. As shown, the Scopus database provided the largest share of studies in our mapping (almost 70% of all included papers), since Scopus aggregates publications across many venues. IEEE Xplore contributed around 10% of the papers, and the ACM Digital Library yielded only one unique paper (<1%). The remaining 20% of papers came from other sources, such as manual searches and reference snowballing beyond the primary databases. This underscores the importance of using multiple search sources and follow-up reference checks to capture all relevant literature.

Table 5. *Distribution of the 132 included papers by source database/origin.*

Source	Papers (N)	% of Dataset

Scopus (journals & conferences)	93	70 %
IEEE Xplore (IEEE publications)	13	10 %
ACM Digital Library (ACM publications)	1	<1 %
Other sources (manual/reference)	25	19 %
Total	**132**	**100 %**

Scopus was the dominant source, while IEEE and ACM digital libraries contributed smaller numbers. The “Other” category includes papers identified through reference tracking and other manual search methods.

4.2 LLM-Supported Traceability Tasks

Large Language Models have been applied to support a variety of traceability link recovery (TLR) tasks in requirements engineering (*see Fig [6]*). The primary tasks observed include linking requirements to source code (REQ–Code), linking requirements to test artifacts (REQ–Test), linking requirements to other requirements (REQ–REQ), and linking requirements to design-level artifacts (REQ–Design). Table [6] summarizes the prevalence of each task in our mapping set. Notably, the **requirements-to-code** traceability task is the most frequently addressed, while **requirements-to-requirements** linking is comparatively rare. Several studies tackle more than one traceability direction – for example, some approaches are evaluated on both requirements–code and requirements–test link recovery – so the percentages sum to over 100%.

Table 6. *LLM-Based Traceability Link Recovery Tasks.*

Traceability Task	**Papers (N)**	**Percentage (%)**
Requirements–Code	50	38 %
Requirements–Design	45	34 %
Requirements–Test	30	23 %
Requirements–Requirement	10	8 %

Requirements–Code (REQ–Code): Linking requirements to source code is the most common TLR application of LLMs. Approximately half of the studies in our dataset (around 50 papers) focus on recovering trace links from textual requirements to corresponding code artifacts (classes, methods, commit messages, etc.), often by exploiting

the semantic understanding of code and natural language provided by pre-trained models. For example, Lin *et al.* [29] compared information retrieval methods and BERT-based deep learning for generating requirement–code links in a bilingual project setting, while Deng *et al.* [30] proposed a multi-task pre-trained model (MTLink) to improve trace link prediction between requirements (in issue descriptions) and code commits. These LLM-based approaches typically outperform earlier keyword-based tracing techniques by capturing semantic similarity between requirement text and source code, thereby improving trace link accuracy.

Requirements–Design (REQ–Design): Linking requirements to design and architectural artifacts is also well-represented, accounting for roughly one-third of the studies (45 papers). LLM-based methods have been used to connect requirements with high-level design models, UML diagrams, architecture documents, and even design decisions. For instance, Marczak-Czajka *et al.* [33] leverage a fine-tuned language model to check whether software design specifications comply with stated requirements and regulatory constraints (tracing requirements to architectural decisions and ensuring alignment). In another example, Timperley *et al.* [34] assess how generative LLMs can assist in creating system architecture designs that satisfy given requirements, effectively bridging the gap from requirements to design solutions. Such approaches demonstrate that LLMs can interpret requirement specifications and map them to design-level concepts or artifacts, supporting activities like compliance verification and model synthesis.

Requirements–Test (REQ–Test): Linking requirements to test cases is addressed in about 20–25% of the publications (approximately 30 papers). In this category, researchers apply LLMs to generate test artifacts from requirements or to recover links between requirement documents and testing assets. A representative example is the work by Zhao *et al.* [31], which uses prompt-driven sequence generation with an LLM to automatically produce acceptance test cases from requirement statements, with each generated test case traceable back to its source requirement. Other studies evaluate off-the-shelf LLMs (e.g. GPT models) for their ability to interpret a requirement and suggest relevant test scenarios or fine-tune models on requirement–test data to improve trace link predictions. Overall, while less common than code tracing, requirement–test traceability is an emerging area where LLMs have shown promise in ensuring that tests adequately cover the specified requirements.

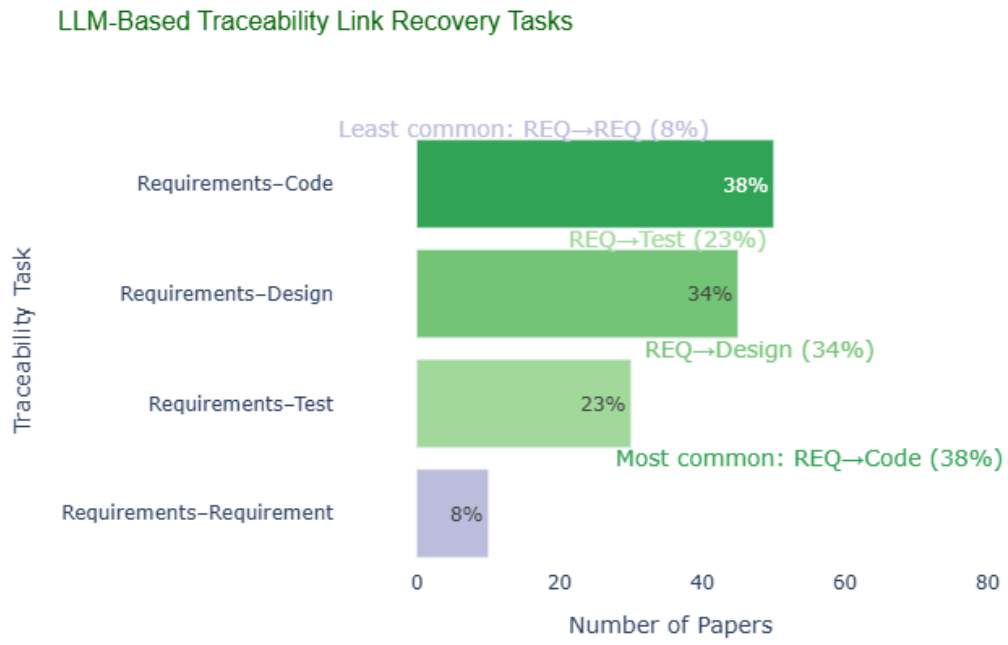

Figure 6. *LLM-Based Traceability Link Recovery Task*

Requirements–Requirement (REQ–REQ): Traceability among requirements (e.g. linking related requirements across documents or detecting duplicates and conflicts) is the least explored of the main tasks, with only a small subset of papers (around 10, roughly 8%) focusing on it. A few studies harness LLMs to identify semantic similarities or inconsistencies between requirements. For example, Fazelnia *et al.* [32] present an approach that uses a satisfiability-aided language model to detect **conflicting requirements** automatically, treating this as a special case of traceability link analysis between requirement pairs. In general, LLM-driven techniques in this category aim to cluster or match requirements that refer to the same feature, or to flag requirement statements that may contradict each other. Although fewer in number, these works highlight the potential of LLMs to support consistency checks and deduplication in requirements engineering by recovering links among related requirements.

4.3 LLM Techniques Used (Prompting, Fine-tuning, RAG)

This section summarizes the primary **techniques used to apply Large Language Models (LLMs)** in the selected studies on traceability link recovery (TLR) as shown in Fig [7]. We classified LLM usage into three major categories: **prompting**, **fine-tuning**, and **retrieval-augmented generation (RAG)**. Several studies used hybrid approaches that fall across multiple categories.

Prompting

Prompting was the most commonly employed technique across the reviewed studies. A total of **58 papers (44%)** used zero-shot, few-shot, or chain-of-thought prompting to instruct a general-purpose LLM (e.g., GPT-3/4, Claude, Gemini) to generate or validate trace links.

Prompting-based methods often involved:

- Supplying requirements as input along with explicit instructions for the LLM to output candidate links
- Designing task-specific prompts to induce the LLM to generate explanations or code snippets
- Using few-shot examples to improve precision or interpretability

For example, **Zhao et al.** [31] used to prompt to generate acceptance test cases traceable to requirements, while **Timperley et al.** [34] employed prompt-based LLMs to support the design of system architectures from requirement inputs. Some studies tested LLM performance with and without in-context examples to assess link quality and generalization.

Fine-tuning

Fine-tuning was applied in **38 studies (29%)**, especially in settings where task-specific training data was available. Fine-tuned models included BERT, CodeBERT, and custom LLM variants adapted to traceability datasets.

This approach allowed researchers to:

- Embed domain-specific patterns into the model weights
- Improve precision and recall for trace link predictions across specific artifact types (e.g., REQ–Code)
- Evaluate performance improvements over zero-shot baselines

For instance, **Deng et al.** [30] introduced **MTLink**, a multi-task fine-tuned model trained on issue and commit traceability data, outperforming IR-based baselines. Similarly, **Lin et al.** [29] compared standard IR with fine-tuned BERT models for bilingual REQ–Code link recovery, showing improved F1-scores with deep learning-based solutions.

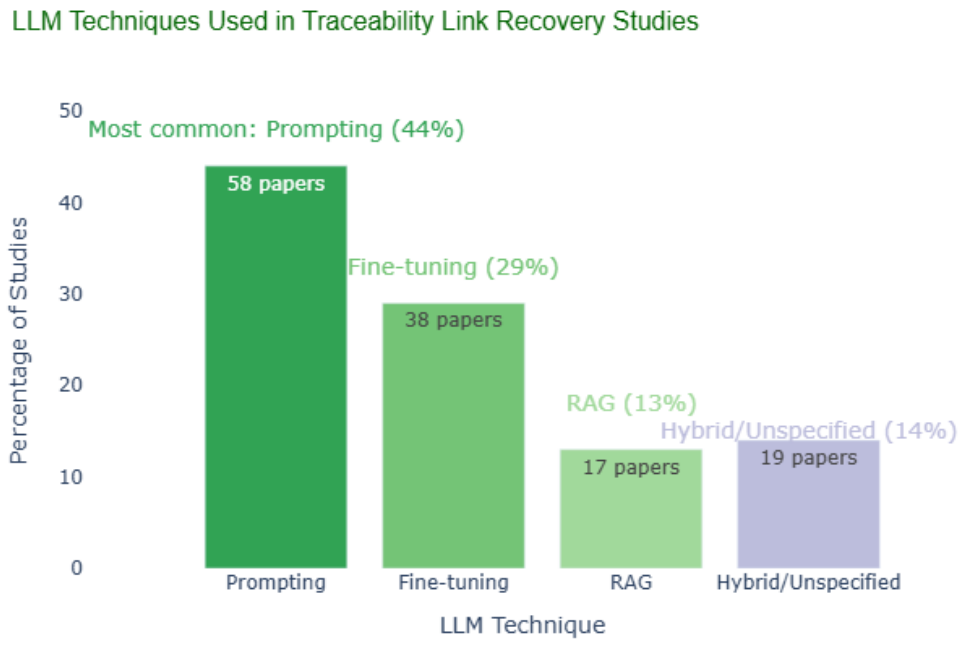

Figure 7. *LLM Techniques used in Traceability Link Recovery*

Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) was used in **17 papers (13%)**, particularly in large-scale or context-sensitive scenarios. RAG-enhanced methods integrate project-specific knowledge bases or document stores to provide additional context to the LLM during inference.

Key benefits of RAG approaches include:

- Reducing hallucination by grounding generation on relevant documents
- Scaling to large corpora of artifacts by selectively retrieving top-k matches
- Handling longer inputs by segmenting and retrieving support passages

A notable example is the work by **Hey et al.** [3], who used RAG with a GPT-based model to recover links between textual requirements, achieving improved trace accuracy compared to standalone generation. Other studies explored hybrid IR + LLM pipelines, where LLMs are used to rerank or refine candidate links generated from retrieval modules.

Hybrid and Unclassified

Approximately **19 papers (14%)** applied hybrid approaches that combined prompting with fine-tuning, or integrated IR pipelines without clear classification. Some works used commercial APIs (e.g., OpenAI, Anthropic) without disclosing exact methods. In these

cases, we categorized them as "hybrid/unspecified" and flagged them accordingly. Table [7] summarizes the results as follows:

Table 7. *Summary Table*

Technique	**Number of Papers**	**Percentage**
Prompting	58	44 %
Fine-tuning	38	29 %
RAG	17	13 %
Hybrid/Unspecified	19	14 %
Total (N = 132)	**132**	**100 %**

4.4 Datasets, Tools, and Evaluation Strategies

A key objective of this mapping study was to understand how researchers evaluate LLM-based approaches to traceability link recovery (TLR). This section summarizes the types of datasets used, the tools or frameworks introduced or evaluated, and the metrics and methods employed to assess performance.

4.4.1 Datasets Used

The studies employed a wide variety of datasets highlighted in Table [8], which we grouped into four broad categories:

Table 8. *Distribution of Dataset Types Used in LLM-Based Traceability Studies*

Dataset Type	**Number of Papers**	**Percentage**
Open-Source Projects	69	52 %
Industrial Datasets	29	22 %
Synthetic Datasets	15	11 %
Benchmark Datasets	19	14 %

Open-source datasets were the most used, appearing in over half of the studies. These included GitHub repositories, Apache projects, and publicly available datasets like iTrust,

eTour, or CoEST. For example, **Lin et al.** [29] and **Zhao et al.** [31] evaluated their methods on multi-language open-source datasets with manually validated links between requirements and code.

Industrial datasets were used in about 22% of the studies. These studies either collaborated with industry partners or analyzed proprietary systems. For example, **Marczak-Czajka et al.** [33] worked on traceability within safety-critical industrial software and used architecture compliance data derived from regulated industry environments. Such studies are valuable for assessing real-world applicability but often suffer from limited reproducibility due to data privacy restrictions.

Synthetic datasets were used in 11% of papers, typically for model training or controlled testing. These are usually small-scale, curated collections where trace links are programmatically defined. They help evaluate models under controlled conditions but may not reflect the noise or complexity of real-world systems.

Finally, **benchmark datasets** appeared in 14% of papers. These datasets were reused across studies to enable comparative evaluation of different LLM approaches as shown in Fig [8]. Popular examples include datasets from the TREC conference or datasets released by earlier TLR studies such as the RETRO or PROMISE datasets.

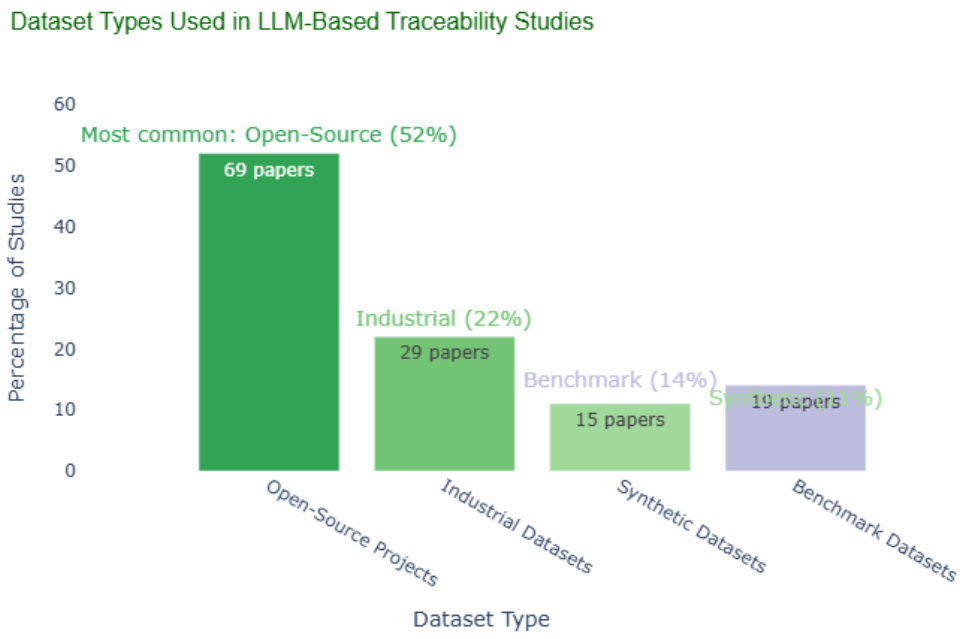

Figure 8. *Dataset Types Used in LLM-Based Traceability Studies*

4.4.2 Tool Support and Frameworks

Out of the 132 studies, **57 papers (43%)** introduced or evaluated a dedicated **tool, framework, or plugin** to support TLR using LLMs. These ranged from standalone Python-based utilities to full-fledged web applications or plugins integrated into development environments.

Examples include:

- **MTLink** [30], a fine-tuned multi-task transformer architecture released as a Python package.
- **Li et al.** [35], who proposed an enhanced prompt-driven reasoning scheme that utilizes LLMs to recover trace links between software artifacts, implemented as a reproducible pipeline for requirements traceability.
- A plugin for VSCode that integrates trace link suggestions into developer workflows (introduced by **Fazelnia et al.** [32]).

Some tools were research prototypes without public release, while others offered GitHub repositories, Docker containers, or notebooks to encourage reproducibility.

4.4.3 Evaluation Strategies and Metrics

Nearly all studies (over 90%) used **quantitative evaluation metrics** to assess TLR performance. The most reported metrics (as shown in Table [9]) were:

Table 9. *Evaluation Metrics Used in LLM-Based Traceability Studies*

Metric	**Usage (% of studies)**
Precision	87 %
Recall	82 %
F1-score	78 %
Accuracy	21 %
BLEU/ROUGE	9 %
Human Evaluation	25 %

Precision, recall, and F1-score were nearly universal for evaluating classification-based TLR methods, especially those using fine-tuned models. For instance, **Deng et al.** [30] reported macro-averaged F1 on their multi-language dataset, while **Zhao et al.** [31] provided detailed comparisons of precision and recall between prompting variants.

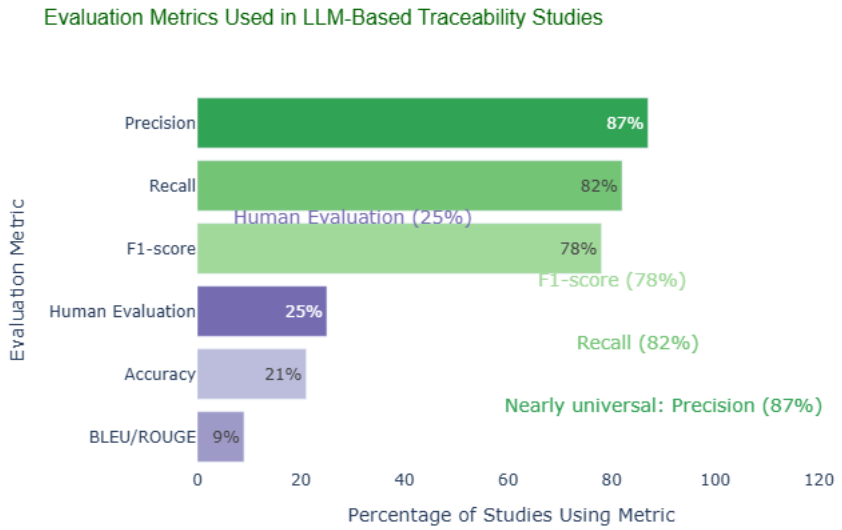

Figure 9. *Evaluation Metrics used in LLM-Based Traceability Tasks.*

A smaller number of studies employed **BLEU or ROUGE** scores, particularly when generating natural language explanations or test cases.

Human evaluation was used in about one-fourth of the studies, either to validate the correctness of trace links or to assess the fluency, plausibility, or usefulness of generated outputs. Human-in-the-loop reviews were particularly important in studies where no ground truth existed or where interpretability was a concern as reflected in Fig [9].

4.5 Reported Challenges and Industrial Use

LLM-based traceability link recovery (TLR) continues to evolve rapidly, but numerous technical and practical limitations are reported in the selected studies. These challenges impact model performance, reliability, interpretability, and real-world integration, particularly in industrial environments.

4.5.1 Technical Challenges in LLM-Based TLR

Hallucination and Semantic Drift:

LLMs occasionally generate trace links that appear plausible but are unsupported or incorrect—commonly referred to as hallucinations. This was a frequent concern, particularly when models lacked task-specific fine-tuning. For example, studies such as [30], [36], and [39] report inconsistent mappings between requirements and code or test artifacts when LLMs overgeneralize from ambiguous input.

Input Length and Context Constraints:

Numerous models are constrained by fixed token limits—typically between 2,048 and 8,192 tokens—which hinders their ability to process lengthy documents or entire code repositories. To address this issue, researchers have employed strategies such as chunking the input or retrieving relevant segments prior to generation; however, these approaches often compromise recall and may introduce selection bias within the context window [38][39].

Domain Mismatch and Incomplete Vocabulary Coverage:

LLMs pre-trained on general corpora such as Common Crawl often struggle to accurately interpret domain-specific terminology. Even state-of-the-art models have been shown to perform poorly on traceability datasets from regulated domains—such as aerospace and finance—unless additional fine-tuning is applied [30].

Explainability and Trust:

The lack of interpretable outputs has been identified as a major barrier to adoption. Engineers evaluating LLM-generated links have specifically requested more transparent explanations or justification snippets, especially when dealing with complex traceability decisions [36].

4.5.2 Challenges in Industrial Deployment

Tool Integration Complexity:

Tools proposed in recent studies required integration with multiple platforms such as GitHub and Jira, leading to challenges related to compatibility and maintainability—particularly in legacy systems that lack a modular architecture [37][40].

Validation and Human Effort:

While automation helps reduce manual effort, human-in-the-loop validation remains crucial. LLM-generated trace links often require significant manual curation, particularly in cases involving ambiguity or loosely defined relationships [36].

Operational and Legal Constraints:

The use of proprietary LLMs in cloud environments has raised concerns regarding data leakage and regulatory compliance, especially within sensitive industrial contexts. Several studies have reported that organizations were reluctant to adopt such models without on-premises deployment options, due to fears of exposing intellectual property [30][37].

4.6 Research Gaps Identified

Although the use of Large Language Models (LLMs) for traceability link recovery (TLR) has gained momentum, the systematic mapping of 132 peer-reviewed studies reveals several persistent and interrelated gaps. These are categorized below into four key dimensions: (1) data and resources, (2) evaluation methodology, (3) application scope and technique coverage, and (4) industrial integration and trust. Each is supported with examples from the studies included.

4.6.1 Limited Availability of Domain-Specific and Public Traceability Datasets

A recurring theme is the lack of standardized and openly accessible datasets suitable for LLM-based traceability research. While some papers used well-known datasets like iTrust or CoEST, most relied on project-specific or manually curated artifacts. Authors stress that the absence of large, diverse, and labeled traceability corpora limits generalization [42]. Similarly, many experiments rely on synthetically linked data, which do not reflect real-world artifact heterogeneity [43].

Even when industrial data is used, it is often confidential and not reproducible. The lack of benchmarking standardization hinders comparative evaluation across tools and models. This gap also inhibits community-driven improvements in LLM fine-tuning or RAG pipelines.

4.6.2 Fragmented Evaluation Metrics and Ground Truth Definitions

Another key issue is the inconsistent use of evaluation metrics across the reviewed studies. While precision, recall, and F1-score dominate, definitions and measurement strategies vary. For example, studies highlight that many evaluations rely on human judgment without inter-rater agreement reporting or annotation protocols [44]. In [41], the authors discuss discrepancies in how a "correct" trace link is defined depending on the context, domain, and abstraction level.

Furthermore, performance comparisons across prompting, fine-tuning, or hybrid approaches are often missing. Only a handful of papers, such as [34], attempted head-to-head comparison of LLM strategies under controlled conditions, and even these lacked detailed ablation analysis.

4.6.3 Narrow Scope of Artifact Types and TLR Directions

Most of the reviewed papers focus on traceability between requirements and source code, with limited attention to other artifact pairs such as requirements–tests, requirements–design, or inter-requirement linking. For example, only a few studies such as [39] explored traceability to test cases, and almost none addressed complex design or model-based engineering artifacts.

Additionally, existing work tends to treat TLR as a static prediction task, rather than integrating it into dynamic, real-time workflows like DevOps, CI/CD pipelines, or agile sprint planning. This overlooks temporal aspects of trace evolution or link decay, as noted in [46].

4.6.4 Limited Industrial Validation and Explainability Support

Although some papers include industrial partners or datasets, comprehensive industrial deployment remains rare. In [38], authors acknowledge that while LLMs produce promising results, practitioners demand transparency, predictability, and model accountability before adoption. Studies reveal that practitioners are hesitant to trust black-box recommendations, especially in safety- or mission-critical environments [36].

Even fewer tools support interactive feedback, confidence calibration, or rationale generation for trace links. In [45], users requested visible trace paths and scoring justifications, indicating a need for more explainable AI (XAI) integration in TLR pipelines.

In sum, this mapping study highlights four central research gaps:

1. **Resource Limitations:** Lack of publicly available, large-scale, and domain-specific traceability datasets.
2. **Evaluation Inconsistency:** Varied metrics, unclear ground truths, and limited benchmarking across methods.
3. **Scope Narrowness:** Overconcentration on REQ–Code links, with underrepresentation of design, test, and evolving trace contexts.
4. **Adoption Barriers:** Weak explainability, insufficient user control, and low maturity of industrial deployments.

Addressing these challenges will be critical for moving LLM-based TLR methods from experimental settings into reliable, scalable, and user-aligned solutions suitable for real-world software engineering environments.

5. DISCUSSION

5.1 Key Insights from the Mapping

The analysis suggests that ***prompting is currently the most common strategy*** for applying LLMs to traceability tasks. This is likely due to its ease of use and the availability of powerful pre-trained models. Fine-tuning is also used, particularly in papers that have access to labeled datasets or where domain-specific customization is needed. The relatively low number of studies using retrieval-augmented generation (RAG) may reflect the complexity of integrating retrieval systems with generative models in software engineering contexts.

Most studies focus on requirements-to-code traceability [47][48][49], possibly because code artifacts are more structured and often better documented. Less attention is given to trace links involving testing, design, or non-functional requirements [35][52]. Only a few works extend to bidirectional or traceability chain scenarios.

Regarding RQ2, prompting-based techniques dominate early LLM applications. Studies such as [40][47][50] leverage in-context prompting on models like GPT-3 or GPT-4. Meanwhile, fine-tuned approaches using domain-specific datasets (e.g., ReLiS, TraceLab) are gaining momentum for higher performance and specificity [38][47][48]. RAG methods, though fewer, show promise in overcoming context length limitations and hallucinations [40][50].

As to RQ3, publicly available datasets such as ReLink, COEST, and eTour are frequently used [30][35][47], often accompanied by precision, recall, and F1-score as evaluation metrics. However, few studies incorporate human validation or external replication settings, which poses challenges for reproducibility.

Finally, in addressing RQ4, several recurring challenges were reported: input length constraints in transformer-based LLMs, hallucination of trace links, lack of domain adaptation, and explainability issues in high-stakes contexts like automotive systems [50]. Privacy concerns were also cited in cloud-based LLM deployment.

5.2 Strengths and Limitations of Current Research

A key strength observed is the diversity of LLM applications across traceability scenarios—from design compliance [50] to rationale-aware trace generation [47]. The field has

also seen a shift toward hybrid and modular LLM pipelines, combining semantic similarity with LLM-based reasoning [30][48].

However, limitations persist:

- Limited industrial-scale validation: Most studies are lab-based or use synthetic datasets. Only a minority report finding from real-world settings [49].
- Tool support is fragmented: While tools like LmRaC [40] and MTLink [30] are proposed, they lack integration with standard RE workflows or lifecycle tools.
- Inconsistency in metrics: Several studies adopt different or even vague evaluation criteria, making cross-comparison difficult [35].

5.3 Implications for Practice and Research

For practitioners, LLMs hold promise for reducing manual overhead in traceability, especially in agile or fast-paced development settings. Yet, adoption requires caution. Factors such as hallucination risks, regulatory explainability, and data privacy must be addressed.

For researchers, the following gaps present opportunities:

- Standardized datasets and benchmarks: The lack of traceability-specific benchmarks limits comparability. Creating open and diversified datasets (e.g., with cross-domain, multilingual artifacts) is crucial.
- Explainable AI (XAI) integration: As shown in Gharbi et al. [52], combining LLMs with explainability layers can bridge the trust gap in safety-critical environments.
- Cost-effective deployment: Cloud-based APIs like GPT-4 incur high costs. Studies must explore on-device, distilled, or open-source LLMs (e.g., LLaMA) for sustainable integration.

5.4 Methodological Limitations

This mapping is limited by its dependence on three digital libraries and keyword-based searches, which may omit relevant grey literature or papers using alternate terminology. The classification process, while systematic, involves interpretation and may introduce subjectivity. Finally, the fast pace of LLM development means some recent work may not yet be indexed in the reviewed databases.

6. CONCLUSION

The increasing complexity of modern software systems has placed renewed emphasis on the need for reliable, scalable, and context-aware requirements traceability. In this thesis, we conducted a systematic mapping study (SMS) to investigate how **Large Language Models (LLMs)** are being used to automate or augment **traceability link recovery (TLR)**—a task that remains both critical and challenging in software engineering.

The study systematically reviewed **132 peer-reviewed publications** published between 2015 and 2025. Using a multi-stage screening process and a classification framework aligned with four research questions, the analysis provides a structured overview of how LLMs are applied to support traceability across various software artifacts, including requirements, source code, test cases, and design models.

Key findings are as follows:

- **RQ1 (Traceability Tasks):** The most addressed TLR task was **requirements-to-code link recovery**, appearing in 38% of the studies. Requirements-to-design and requirements-to-test were also frequently explored, while requirements-to-requirements links received comparatively less attention. This reflects the practical emphasis on downstream traceability and automation of validation workflows.
- **RQ2 (LLM Techniques):** Prompting emerged as the dominant method (44%), followed by fine-tuning (29%) and retrieval-augmented generation (13%). While prompting is accessible and model-agnostic, fine-tuning tends to offer superior accuracy when task-specific datasets are available. RAG approaches are increasingly explored for handling context-rich environments.
- **RQ3 (Datasets and Evaluation):** Over half of the studies used open-source datasets, while roughly one-fifth relied on industrial data. Evaluation strategies predominantly relied on F1-score, precision, and recall, with about a quarter of the studies incorporating human review. Tool support was present in 43% of the studies, often in the form of research prototypes or command-line utilities.
- **RQ4 (Challenges and Industrial Use):** Despite encouraging results, the review revealed several challenges. These include **input length limitations**, **model hallucination**, **lack of domain adaptation**, and **limited generalizability across artifact types**. Industrial adoption remains relatively limited, though some studies demonstrate promising collaborations or evaluations in real-world settings.

Contributions and Implications

This thesis offers several contributions to the fields of requirements engineering and AI-assisted software development:

1. A structured synthesis of the state of the art in LLM-based traceability.
2. A thematic classification framework that can guide future empirical studies.
3. Identification of under-explored traceability tasks and methodological gaps.
4. Practical insights into evaluation practices, tool readiness, and industrial applicability.

For practitioners, the findings suggest that LLMs—especially when coupled with retrieval mechanisms or fine-tuned architectures—can enhance the automation of trace link generation. However, accuracy, explainability, and scalability remain important considerations.

Limitations

As with any mapping study, this thesis is limited by its scope and selection boundaries. Only peer-reviewed English-language papers were considered, and although multiple databases were used (Scopus, IEEE Xplore, ACM DL), some relevant studies may still have been missed. Furthermore, classification relied on available information, and not all studies provided complete reporting on datasets or evaluation methods.

Future Work

Future research may extend this study in several directions:

- **Meta-analysis** of model performance across common datasets to quantify effectiveness;
- Development of **benchmark suites** tailored for LLM-based TLR methods;
- Investigation into **multi-agent architectures** or **interactive systems** where LLMs collaborate with human analysts;
- Exploration of **traceability in emerging domains**, such as ML pipelines, cybersecurity, or sustainability requirements.

Additionally, there is a clear opportunity for research on improving the **trustworthiness, explainability, and regulatory alignment** of LLMs when used in safety-critical or compliance-driven settings.

Final Remark

In conclusion, this thesis confirms that Large Language Models are beginning to reshape how traceability is understood and implemented in software engineering. While much progress has been made, meaningful challenges remain. This mapping provides a foundation for advancing both the academic understanding and practical integration of LLMs into traceability workflows.

7. REFERENCES

[1] **Fuchß, D., Hey, T., Keim, J., & Koziolek, A. (2025).** LiSSA: Toward generic traceability link recovery through retrieval-augmented generation. *In Proceedings of the 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)* (pp. 1396–1408). IEEE. https://doi.org/10.1109/ICSE55347.2025.00186

[2] **George, M., Fischer-Hellmann, K.-P., Knahl, M., Bleimann, U., & Atkinson, S. (2012).** Traceability in model-based testing. *Future Internet, 4*(4), 1026–1036. https://doi.org/10.3390/fi4041026

[3] **Hey, T., Fuchß, D., Keim, J., & Koziolek, A. (2025).** *Requirements Traceability Link Recovery via Retrieval-Augmented Generation*. In *International Working Conference on Requirements Engineering: Foundation for Software Quality (REFSQ 2025)* (Lecture Notes in Computer Science, Vol. 15588, pp. 381–397). Springer Cham. https://doi.org/10.1007/978-3-031-88531-0_27

[4] **Marcén, A. C., Lapeña, R., Pastor, Ó., & Cetina, C. (2020).** Traceability link recovery between requirements and models using an evolutionary algorithm guided by a learning to rank algorithm: Train control and management case. *Journal of Systems and Software, 163*, 110519. https://doi.org/10.1016/j.jss.2020.110519

[5] **Zhao, W. X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., Min, Y., Zhang, B., Zhang, J., Dong, Z., Du, Y., Yang, C., Chen, Y., Chen, Z., Jiang, J., Ren, R., Li, Y., Tang, X., Liu, Z., Liu, P., Nie, J.-Y., & Wen, J.-R. (2025).** A survey of large language models. *arXiv preprint arXiv:2303.18223*. https://arxiv.org/abs/2303.18223

[6] **Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017).** Attention is all you need. *arXiv preprint arXiv:1706.03762*. https://arxiv.org/abs/1706.03762

[7] **Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019).** BERT: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*. https://arxiv.org/abs/1810.04805

[8] **Gotel, O. C. Z., & Finkelstein, C. W. (1994).** An analysis of the requirements traceability problem. *In Proceedings of the IEEE International Conference on Requirements Engineering* (pp. 94–101). IEEE. https://doi.org/10.1109/ICRE.1994.292398

[9] **Cleland-Huang, J., Gotel, O. C. Z., Hayes, J. H., Mäder, P., & Zisman, A. (2014).** Software traceability: Trends and future directions. *In Proceedings of the Future of Software Engineering (FOSE 2014)* (pp. 55–69). Association for Computing Machinery. https://doi.org/10.1145/2593882.2593891

[10] **ISO/IEC/IEEE. (2018).** *ISO/IEC/IEEE 29148:2018(E) – Systems and software engineering — Life cycle processes — Requirements engineering* (pp. 1–104). https://doi.org/10.1109/IEEESTD.2018.8559686

[11] **Mäder, P., & Cleland-Huang, J. (2010, October).** A visual traceability modeling language. *In D. Petriu, N. Rouquette, & Ø. Haugen (Eds.), Model Driven Engineering Languages and Systems – 13th International Conference, MODELS 2010, Oslo, Norway, October 3–8, 2010, Proceedings, Part I* (pp. 226–240). Springer. https://doi.org/10.1007/978-3-642-16145-2_16

[12] **Guo, J., Cheng, J., & Cleland-Huang, J. (2017, May).** Semantically enhanced software traceability using deep learning techniques. *In 2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)* (pp. 3–14). IEEE. https://doi.org/10.1109/ICSE.2017.9

[13] **Zhu, J., Xiao, G., Zheng, Z., & Sui, Y. (2022).** Enhancing traceability link recovery with unlabeled data. *In Proceedings of the 2022 IEEE 33rd International Symposium on Software Reliability Engineering (ISSRE)* (pp. 446–457). IEEE. https://doi.org/10.1109/ISSRE55969.2022.00050

[14] **Antoniol, G., Canfora, G., Casazza, G., De Lucia, A., & Merlo, E. (2002).** Recovering traceability links between code and documentation. *IEEE Transactions on Software Engineering, 28*(10), 970–983. https://doi.org/10.1109/TSE.2002.1041053

[15] **Marcus, A., & Maletic, J. I. (2003).** Recovering documentation-to-source-code traceability links using latent semantic indexing. *In Proceedings of the 25th International Conference on Software Engineering (ICSE)* (pp. 125–135). IEEE. https://doi.org/10.1109/ICSE.2003.1201194

[16] **Lin, J., Liu, Y., Zeng, Q., Jiang, M., & Cleland-Huang, J. (2021).** Traceability transformed: Generating more accurate links with pre-trained BERT models. *arXiv preprint arXiv:2102.04411*. https://arxiv.org/abs/2102.04411

[17] **Ali, N., Guéhéneuc, Y.-G., & Antoniol, G. (2013).** Trustrace: Mining software repositories to improve the accuracy of requirement traceability links. *IEEE Transactions on Software Engineering, 39*(5), 725–741. https://doi.org/10.1109/TSE.2012.71

[18] **Mills, C., Escobar-Avila, J., & Haiduc, S. (2018).** Automatic traceability maintenance via machine learning classification. *In Proceedings of the 2018 IEEE International Conference on Software Maintenance and Evolution (ICSME)* (pp. 369–380). IEEE. https://doi.org/10.1109/ICSME.2018.00045

[19] **Ruan, H., Chen, B., Peng, X., & Zhao, W. (2019).** DeepLink: Recovering issue-commit links based on deep learning. *Journal of Systems and Software, 158*, 110406. https://doi.org/10.1016/j.jss.2019.110406

[20] **Alor, E., Khatoonabadi, S., & Shihab, E. (2025).** Evaluating the use of LLMs for documentation to code traceability. *arXiv preprint arXiv:2506.16440*. https://arxiv.org/abs/2506.16440

[21] **Huang, H., Widyasari, R., Zhang, T., Irsan, I. C., Shi, J., Ang, H. W., Liauw, F., Ouh, E. L., Shar, L. K., Kang, H. J., & Lo, D. (2025).** Back to the basics: Rethinking issue-commit linking with LLM-assisted retrieval. *arXiv preprint arXiv:2507.09199*. https://arxiv.org/abs/2507.09199

[22] **Petersen, K., Feldt, R., Mujtaba, S., & Mattsson, M. (2008).** Systematic mapping studies in software engineering. *In Proceedings of the 12th International Conference on Evaluation and Assessment in Software Engineering (EASE'08)* (pp. 68–77). BCS Learning & Development Ltd.

[23] **Kitchenham, B. A. (2012).** Systematic review in software engineering: Where we are and where we should be going. *In Proceedings of the 2nd International Workshop on Evidential Assessment of Software Technologies (EAST '12)* (pp. 1–2). Association for Computing Machinery. https://doi.org/10.1145/2372233.2372235

[24] **Garousi, V., & Mäntylä, M. V. (2016).** Citations, research topics and active countries in software engineering: A bibliometrics study. *Computer Science Review, 19*, 56–77. https://doi.org/10.1016/j.cosrev.2015.12.002

[25] **Hannousse, A., & Yahiouche, S. (2021).** Securing microservices and microservice architectures: A systematic mapping study. *Computer Science Review, 41*, 100415. https://doi.org/10.1016/j.cosrev.2021.100415

[26] **Kitchenham, B., & Charters, S. (2007, July 9).** *Guidelines for performing systematic literature reviews in software engineering* (EBSE Technical Report EBSE-2007-01). Keele University & Durham University. https://www.elsevier.com/__data/promis_misc/525444systematicreviewsguide.pdf

[27] **Kitchenham, B. A., Budgen, D., & Brereton, O. P. (2011).** Using mapping studies as the basis for further research – A participant-observer case study. *Information and Software Technology, 53*(6), 638–651. https://doi.org/10.1016/j.infsof.2010.12.011

[28] **Budgen, D., Turner, M., Brereton, P., & Kitchenham, B. A. (2008).** Using mapping studies in software engineering. *Proceedings of the PPIG 2008 Workshop*. https://www.researchgate.net/publication/228573714_Using_Mapping_Studies_in_Software_Engineering

[29] **Lin J., Liu Y., Cleland-Huang J. (2022).** *Information retrieval versus deep learning approaches for generating traceability links in bilingual projects*. Empirical Software Engineering **27**: 97. DOI: 10.1007/s10664-021-10050-0

[30] **Deng Y., Wang B., Zhu Q., et al. (2024).** MTLink: Adaptive multi-task learning based pre-trained language model for traceability link recovery between issues

and commits. J. King Saud Univ. – Computer & Information Sciences (in press). DOI: 10.1016/j.jksuci.2024.101958

[31] **Zhao, G., Fang, G., Du, D., Gao, Q., & Zhang, M. (2023).** Leveraging conditional statement to generate acceptance tests automatically via traceable sequence generation. *In 2023 IEEE 23rd International Conference on Software Quality, Reliability, and Security (QRS)* (pp. 394–405). IEEE. https://doi.org/10.1109/QRS60937.2023.00046

[32] **Fazelnia, M., Mirakhorli, M., & Bagheri, H. (2024).** Translation titans, reasoning challenges: Satisfiability-aided language models for detecting conflicting requirements. *In Proceedings of the 2024 IEEE/ACM International Conference on Automated Software Engineering (ASE 2024)* (pp. 2294–2298). Association for Computing Machinery. https://doi.org/10.1145/3691620.3695302

[33] **Marczak-Czajka, A., Dearstyne, K., & Cleland-Huang, J. (2025).** Assessing compliance of software system designs to laws, regulations, and their underlying values. *In Proceedings of the 2025 IEEE/ACM International Workshop on Designing Software (Designing)* (pp. 47–54). IEEE. https://doi.org/10.1109/Designing66910.2025.00015

[34] **Timperley, L. R., Berthoud, L., Snider, C. M., & Tryfonas, T. (2025).** Assessment of large language models for use in generative design of model-based spacecraft system architectures. *Journal of Engineering Design*. Advance online publication. https://doi.org/10.1080/09544828.2025.2453401

[35] **Li, Y., Zhang, R., & Liu, J. (2024).** An enhanced prompt-based LLM reasoning scheme via knowledge graph-integrated collaboration. *In Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*. Springer. https://doi.org/10.1007/978-3-031-72344-5_17

[36] **Crivellari, A., Resch, B., & Shi, Y. (2022).** TraceBERT—A feasibility study on reconstructing spatial–temporal gaps from incomplete motion trajectories via BERT training process on discrete location sequences. *Sensors, 22*(4), 1682. DOI: 10.3390/s22041682

[37] **Wang, Y., Hu, K., Jiang, B., Xia, X., & Tang, X.-S. (2023).** A systematic literature review of software traceability links automation techniques. *Chinese Journal of Computers), 46*(9), 1919–1946. https://doi.org/10.11897/SP.J.1016.2023.01919 (Scopus link: https://www.scopus.com/pages/publications/85173597434?inward)

[38] **Shaikh, R., Malekar, M., & Kumar, R. (2025).** Coding Agents: A comprehensive survey of automated software development. *In Proceedings of CSNT 2025*. IEEE. https://doi.org/10.1109/CSNT64827.2025.10968728

[39] **Y. Gao, L. Yu, Y. Shi, et al. (2024).** Leveraging large language models for efficient software traceability. *Electronics, 13*(22), 4425. DOI: 10.3390/electronics13224425

[40] **Craig, D. B., & Drăghici, S. (2024).** LmRaC: A functionally extensible tool for LLM interrogation of user experimental results. *Bioinformatics, 40*(12), btae679. DOI: 10.1093/bioinformatics/btae679

[41] **Yang, H., Ji, S., Wu, R., & Xu, W. (2024).** Are you being tracked? Discover the power of zero-shot tracing with LLMs. *In Proceedings of the 2024 IEEE International Conference on Cyber Security, Cyber Warfare and Digital Forensics (CSCAIoT)*. IEEE. https://doi.org/10.1109/CSCAIoT62585.2024.00008

[42] **Tian, J., Zhang, L., & Lian, X. (2023).** A cross-level requirement trace link update model based on bidirectional encoder representations from transformers. *Mathematics, 11*(3), Article 623. DOI: 10.3390/math11030623

[43] **Wang, L., Wang, M.-C., Zhang, Y.-R., Ma, J., Shao, H.-Y., & Chang, Z.-X. (2025).** Automated identification and representation of system requirements based on large language models and knowledge graphs. *Applied Sciences, 15*(7), 3502. DOI: 10.3390/app15073502

[44] **Zadenoori, M. A., Zhao, L., Alhoshan, W., & Ferrari, A. (2025).** Automatic prompt engineering: The case of requirements classification. In **A. Hess & A. Susi (Eds.)**, *Requirements Engineering: Foundation for Software Quality* (REFSQ 2025, Lecture Notes in Computer Science, Vol. 15588, pp. 217–225). Springer, Cham. https://doi.org/10.1007/978-3-031-88531-0_15

[45] **Wu, K., Pang, L., Shen, H., Cheng, X., & Chua, T.-S. (2023).** LLMDet: A third-party large language models generated text detection tool. *Findings of the Association for Computational Linguistics: EMNLP 2023* (pp. 2113–2133). Association for Computational Linguistics. DOI: 10.18653/v1/2023.findings-emnlp.139

[46] **Warren, G., Shklovski, I., & Augenstein, I. (2025).** Show Me the Work: Fact-Checkers' requirements for explainable automated fact-checking. *Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI '25)*, Paper 3713277. Association for Computing Machinery. DOI: 10.1145/3706598.3713277

[47] **Lan, J., Gong, L., Zhang, J., & Zhang, H. (2023).** BTLink: Automatic link recovery between issue reports and code commits using BERT. *Empirical Software Engineering, 28*(4), Article 103. DOI: 10.1007/s10664-023-10342-7

[48] **Zhu, J., Xiao, G., Zheng, Z., & Sui, Y. (2024).** Deep semi-supervised learning for recovering traceability links between issues and commits. *Journal of Systems and Software, 206*, 112109. https://doi.org/10.1016/j.jss.2024.112109

[49] **Habib, A., Bano, M., & Khan, S. (2024).** Traceability challenges in industrial software projects: A case study in the automotive domain. *Information and Software Technology*. https://doi.org/10.1016/j.infsof.2024.107439

[50] **Maro, S., Steghöfer, J.-P., & Staron, M. (2018).** Software traceability in the automotive domain: Challenges and solutions. *Journal of Systems and Software, 141*, 85–110. https://doi.org/10.1016/j.jss.2018.03.060

[51] **Chen, C., Zhang, Z., Khalilov, I., Guo, B., Gebreegziabher, S. A., Ye, Y., Xiao, Z., Yao, Y., Li, T., & Li, T. J.-J. (2025).** Toward a human-centered evaluation framework for trustworthy LLMpowered GUI agents (arXiv:2504.17934v2). *arXiv*. https://doi.org/10.48550/arXiv.2504.17934

APPENDIX A:

1. SLR Log Sheet contains: Titles, authors, and publication metadata of all retrieved studies. Inclusion and exclusion decisions with justifications. Screening outcomes for each stage (title, abstract, full text). Notes from reviewers.

   https://docs.google.com/spreadsheets/d/1uX8B615NbJuItgU_2BPCT61WA-ebLVMZ/edit?usp=sharing&ouid=103675061687575260525&rtpof=true&sd=true